

Content-based 3D shape retrieval system

Thomas van Deukeren (5721490), Gerard van Schie (6585442)

Utrecht University

Saturday 21st February, 2026

Abstract

In this paper we present a full retrieval system able to be used on 3D shape databases. The shapes are presented as points in a high-dimensional space, which are in turn able to be quickly matched with the query shape a user provides. Two different query methods are covered. A custom query method which gives a lot of flexibility in the metrics that can be used and a query method aimed to support large database systems. The performance of the system is both query systems promising, returning approximately 7 out of 10 query shapes with the same category. The average query time for the scalable query method is $8.20345 * 10^{-4}$ seconds. A set of 380 shapes from a well known database is used for visualisations and evaluation.

1. Introduction

Multi-dimensional objects are commonplace in our society, much more than most people might realize. From well known sources like movies and videogames to lesser ones like prosthetics adapted to a patients unique physiology. Not to mention how much 3D printing has grown in popularity and ease of access in recent years. A side-effect of this is that we have many different 3D objects with many different shapes and qualities. This leaves us with the question, how can we organize these efficiently?

The first possible solution for this problem would be to organize them manually. It is easy for us to distinguish one 3D object from another, but a problem is that when presented with thousands of objects, a human will require too much time to 'process' them all. An obvious solution to this problem is to let a program solve this problem. Computer programs are able to distinguish 2D shapes relatively easily already, but when presented with 3D objects, the process becomes significantly more complex.

This paper presents our attempt to create a 3D data retrieval system. During the creation of this system, we used our own systems and publicly available 3D shape databases. We were unable to test the system on databases larger than 380 and were given roughly two months time to complete it.

1.1. Development tools

The data retrieval system is created using Python[**Python**] because there is a lot of libraries available that can be used for the end-to-end system. To load, save, re-sample, re-mesh, and manipulate the meshes we use Open3D[**Package _ Open3D**]. In addition, this library contains a lot of functionalities for extracting features from 3D shapes and uses parallelism to run efficiently. However, because graphical user information (GUI) functionality of has only been added recently we use a more matured library PyQt6[**Package _ PyQt6**] to create the front-end. The latest supported Python version for these two libraries at the time of development was 3.9. Finally, for creating figures we have used matplotlib[**Matplotlib**].

1.2. 3D shape definitions

A 3D shape (mesh) $M = \{c_i\}$ is defined by a set of n non-overlapping cells c_i . A cell $c = \{v_j\}$ is defined by a list of points $v_j = \{x_j, y_j, z_j\}$, these points are also called vertices. There are a total of m vertices that define a mesh. Cells can be both triangles and quads defined by respectively 3 or 4 cell vertices. The coordinates of the vertices of a quad are not restricted to only rectangular. A point cloud $P = \{v_j\}$ is a different representation of a 3D shape because it only contains the vertices. Finally, an Axis Aligned Bounding Box (AABB) is the smallest box the mesh fits in with the edges parallel to the axis.

1.3. Shape descriptors

There are several levels of detail in describing the characteristics of a shape, also called attributes. Global examples which are on the mesh level are the number of holes, and the category the mesh belongs to. An example of an attribute on cell level is the number of corners describing that cell or the color. A vertex can also have attributes, their distance to other vertices or the number of cells attached to the vertex.

1.4. Report structure

The structure of the report is a step-wise process to creating a multimedia retrieval system for 3D shapes. At first the database will be discussed and information will be visualized. The normalization will be explained and verified in detail. Then features are covered, after which querying is explained. At the end of the report the performance of the system will be evaluated.

2. Step 1: Read and view the data

This section aims to give information about the types of shapes in the data. How this data is handled and gives some visual examples of the data.

2.1. Data description

For the data retrieval system, we selected the Labeled PSB Dataset[**Dataset_PSB**]. It contains 380 meshes divided into 19 categories, each containing 20 meshes, in Object File Format (OFF). Examples of categories are Airplane, Octopus, Fish, and Table. Each mesh also comes with a small image visualizing the mesh. We chose this dataset since it was relatively small and does not contain many different mesh categories. So if the database has to get reduced we can still ensure each category will contain a reasonable number of shapes. The system can be extended with other datasets to have a greater set to query from.

2.2. Loading data

The retrieval system handles both Polygon File Format (PLY) and OFF. Both these formats contain a list of vertices in combination with a list of triangles and/or quads with the index of their 3 or 4 vertices in the initial list. Because the PLY format is better supported in Open3D the OFF files in the dataset are first converted to PLY format and then loaded. Loading the PLY and OFF formats and writing them to the PLY format is done using the read and write functionality of Open3D. The retrieval system will only handle PLY files internally.

2.3. Visualization of data

A basic GUI (Figure 1) is created to visualize a mesh, so it can be visually inspected.

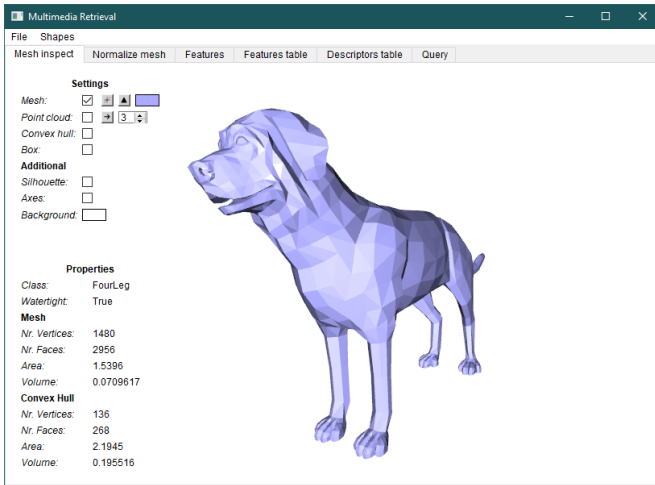


Figure 1: Showcase of the GUI

Such an inspector tool is often referred to as a PLY viewer. For this, the Open3D visualizer window is used and put as a widget inside the GUI. By dragging the mouse over the window, the user can rotate the camera around the mesh. Zooming in and out is done using the mouse wheel (Figure 2). Translating (Figure 3) or rotating is done by dragging the mouse over the screen whilst holding the control or shift key. These are all functionalities of the Open3D window itself.

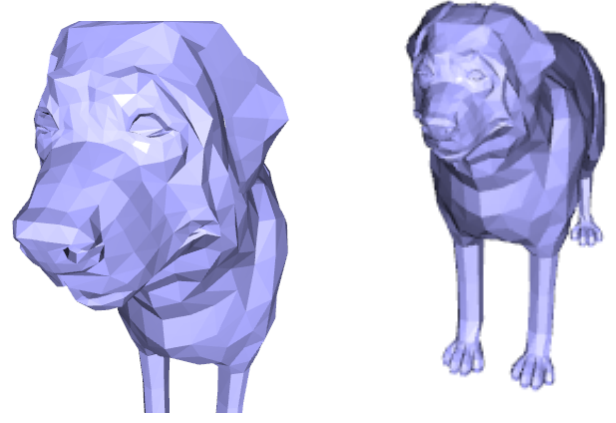


Figure 2: Showcase of the zoom functionality in the PLY viewer

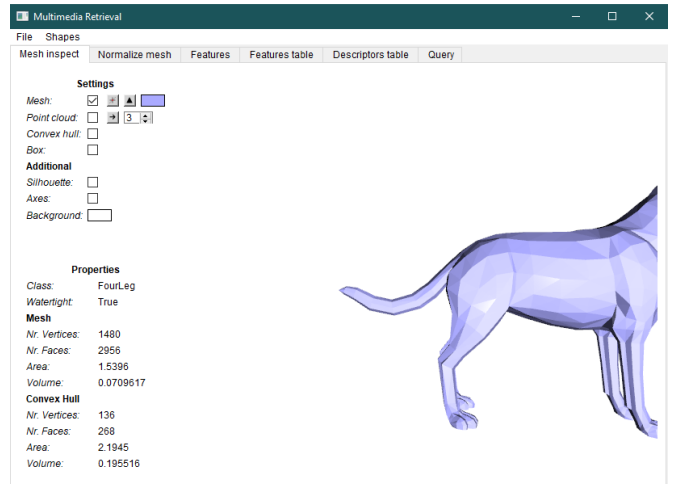
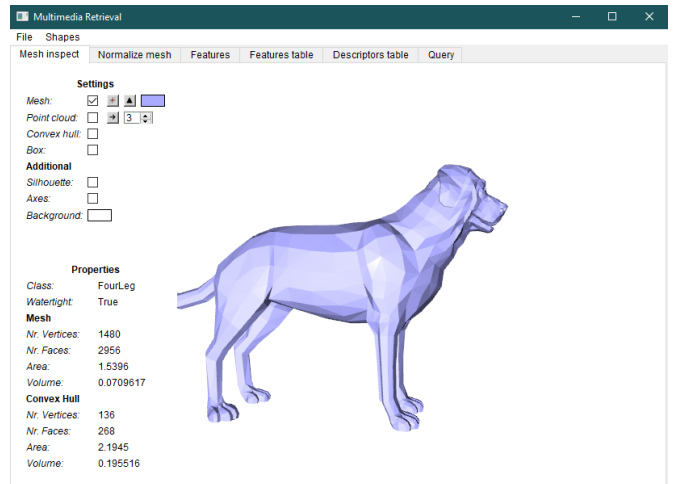


Figure 3: Showcase of the translate functionality in the PLY viewer

3. Step 2: Preprocessing and cleaning

This section aims to create a shape database and visualize the extracted features of the dataset. A start will be made on normalization, which aims to remove unwanted differences in shapes before feature extraction is done.

The main reason for normalization is feature extraction. For example, extracting the surface area of meshes

with different scales results in very different values for the same shapes. The resampling needs to be done so alignment, which uses a statistical method, can give good results even if the original mesh has few vertices.

3.1. Step 2.1: Analyzing a single shape

Initially, a few features will be extracted from the shapes. These features are the class of the shape, for which the selected dataset is the name of the parent folder in which the file is located. Additionally, the number of faces and vertices are extracted, which are listed in the left panel of the PLY viewer, as shown in Figure 1. The type of faces and the axis-aligned 3D box which is visible in the PLY viewer window.

Open3D loads the PLY file and creates a TriangleMesh object. This class can only contain vertices and triangles, so quads are converted to triangles automatically. For that reason, the type of faces will always be 'only triangles'. So the 'type of faces' feature will not be included in the application or database.

The features are extracted from the shapes and written to a comma-separated values (CSV) file. This is a common file format in which values are separated by commas. Each line, except the header, in the CSV file contains the features of a single mesh. Starting with the path to the mesh as the unique identifier. Then the features will be listed in that line. When loading in the meshes, the path to the file is checked in the database. If the path of the mesh already exists in the database, then the feature attributes are immediately populated.

A couple of entries of the database are shown in Table 1. These features are computed for all shapes in the dataset.

3.2. Step 2.2: Statistics over the whole database

One of the first things that we noticed when looking at the features was that the histograms of the vertices and the faces were very similar, as shown in Figure 4 and Figure 5. The number of faces is also roughly twice the number of vertices, rather than three times, which might sound more intuitive as you need three vertices to create a triangle. This is however characteristic of Euler's formula for triangle meshes, which states that a triangle mesh has roughly twice the number of faces than vertices.

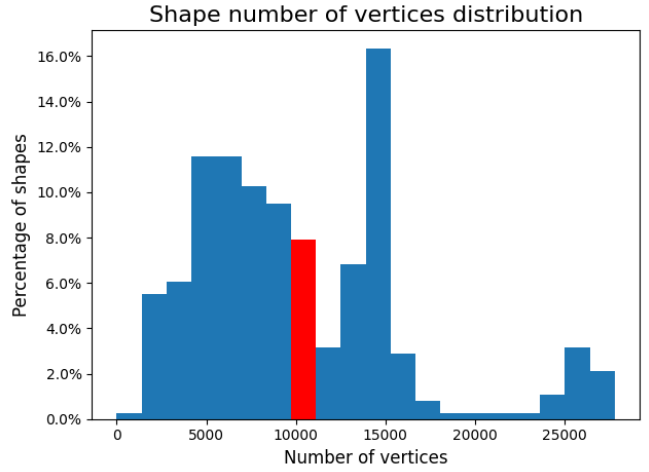


Figure 4: The histogram of all the Shapes in the database when looking at the number of vertices. The red bar is the average value of the data.

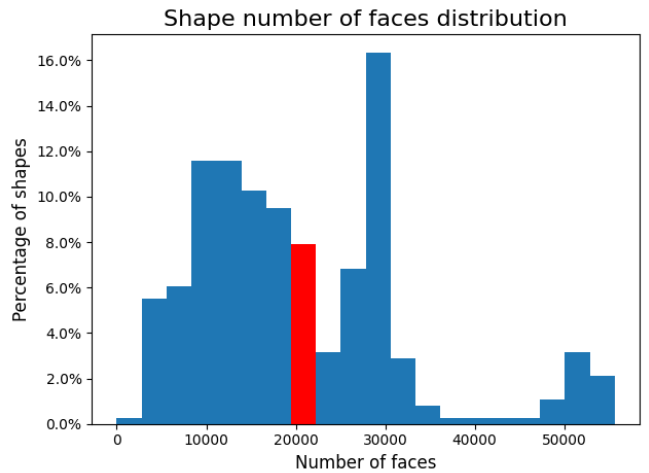


Figure 5: The histogram of all the Shapes in the database when looking at the number of faces. The red bar is the average value of the data.

The red bars in the histograms will not be stated in later figures.

Our first attempt to get the average Shape from the database, based on a given feature, involved using the median of that feature list. However, when numpy's median function calculates a new value due to an odd number of values, then the Shape of this new value does not exist.

We now retrieve average Shapes using the mathematical average of all the values in the feature list and taking the closest Shapes to that average.

Path	Class	Nr. Faces	Nr. Vertices	AABB p0	AABB p1
['Airplane' '61.ply']	Airplane	5400	10796	[-0.475 -0.976 -0.212]	[0.602 0.990 0.119]
['Airplane' '62.ply']	Airplane	5619	11234	[-0.801 -0.154 -0.931]	[0.811 0.292 0.999]
['Airplane' '63.ply']	Airplane	5519	11034	[-0.995 -0.124 -0.845]	[1.000 0.278 0.853]
['Airplane' '64.ply']	Airplane	6797	13590	[-0.768 -0.840 -0.146]	[0.772 1.000 0.344]
['Airplane' '65.ply']	Airplane	6448	12892	[-1.000 -0.856 -0.149]	[0.695 0.868 0.214]
['Airplane' '66.ply']	Airplane	7739	15474	[-0.997 -0.738 -0.104]	[0.686 0.738 0.219]

Table 1: A simplified overview of database entries stored in CSV file format. The AABB is stored by using two diagonal vertices of the box. p0 is the point with the minimum x, y, and z whilst p1 is the vertex with the maximum coordinates.

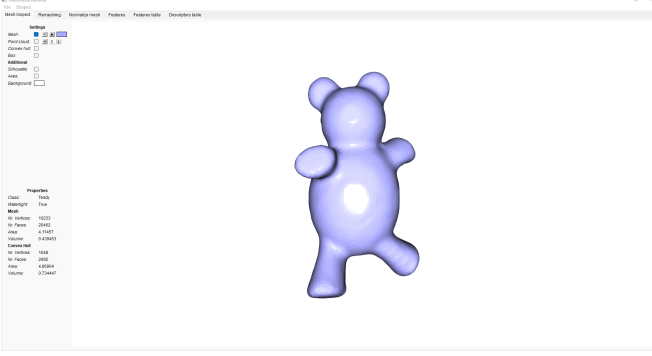


Figure 6: The Shape closest to the average of all Shapes when looking at the number of vertices and faces.

For the averages and outliers, we have automated the process of writing their Shape data to their own txt files.

To calculate any outliers, we used matplotlib's boxplot and the 1.5 IQR rule. When applied to the vertices and faces features, we detect two outliers, as shown in Figure 8 and Figure 9, and get the boxplot as shown in Figure 7.

Detected 2 outliers in 380 Shapes when looking at the nr_vertices feature

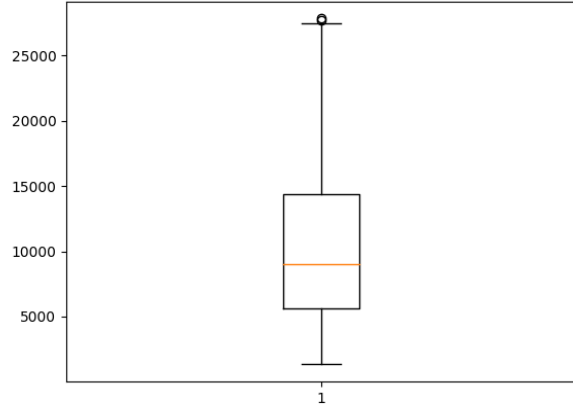


Figure 7: The boxplot of the outliers of the vertices (and faces) feature

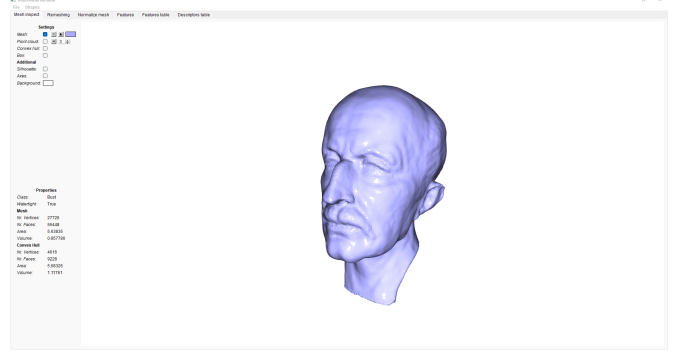


Figure 8: The outlier of the vertices and faces feature with the lowest value

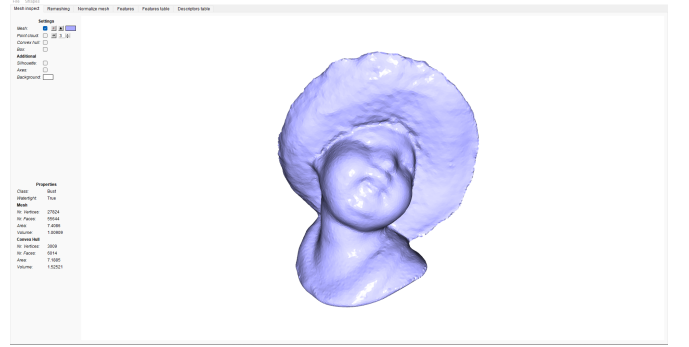


Figure 9: The outlier of the vertices and faces feature with the highest value

3.3. Ordering of the normalization steps

To explain the ordering of the steps, we will go through them in reverse order, showing for each step that it only depends on previous steps in the normalization pipeline. After describing the ordering, each step is explained in detail. The pipeline is as follows:

1. Number of vertices: Give all meshes the same 'resolution'
2. Translation: Centring the mesh to the origin.
3. Alignment: Orient the mesh to have a consistent view angle.
4. Flipping: Flip mesh along its axis to prevent mirrored shapes
5. Scale: All meshes have the same size.

The goal of scaling is to fit the mesh inside a unit cube. The maximum and minimum x, y, and z coordinates are used to determine the scalar. Aligning the mesh changes the maximum and minimum coordinates. So that step is done prior to scaling.

Flipping the mesh is based on the moment test, it rotates the mesh along the axis to have the majority of

the weight on the left. Alignment after this step could make it so that the majority of the weight is on the right side again, so this step is done after alignment.

Alignment describes along which directions the points are spread the most. The method used, Principal component analysis (PCA), uses the origin of the coordinate system as the center along which the points are spread. The 3 vectors it creates for 3 dimensional data are all perpendicular to each other. Before PCA, it is convenient to translate the mesh so the center of the mesh corresponds to the center of the coordinate system.

Translation to the center of the coordinate system depends on the average coordinate of the points in the mesh. Changing the number of vertices and the spread of the vertices shifts the origin of the mesh. So normalizing the vertices is done before this step.

3.4. Step 2.3: Resampling outliers

For the normalization we aimed at creating 10000 vertices, this would still preserve most of the details, whilst still allowing for fast enough feature extraction in later steps. The method used is Poisson disk sampling [PoissonDiskSampling]. However, this method creates a point cloud instead of a mesh.

When reconstructing a mesh from a point cloud there were different types of issues. The first was that cells were missing from the eventual mesh, so parts of the shape did not get reconstructed. Another problem were weird artifacts that would be created which were not in the original shape, especially around thin sections of the shape. So instead, the normalization steps are performed on the initial mesh based on values of the point cloud representation. So the transformations are performed in parallel on both the mesh and point cloud representation.

The initial shape would not always have cells of equal area. In later steps a sampling method is used which takes a uniform spread over the surface of the shape.

3.5. Step 2.4: Checking the resampling

Because the mesh is not reconstructed from the point cloud, the number of vertices is the same before and after step 2.3. Instead we will first look at an example of one of the point clouds with 10000 vertices of which the surface has been reconstructed.

Instead, we will give a visual result and plots to show the quality of Poisson disk sampling gives. This quality can be checked by trying to reconstruct the surface from the point cloud and showing the uniformity of the cell areas. As can be seen in Figure 10

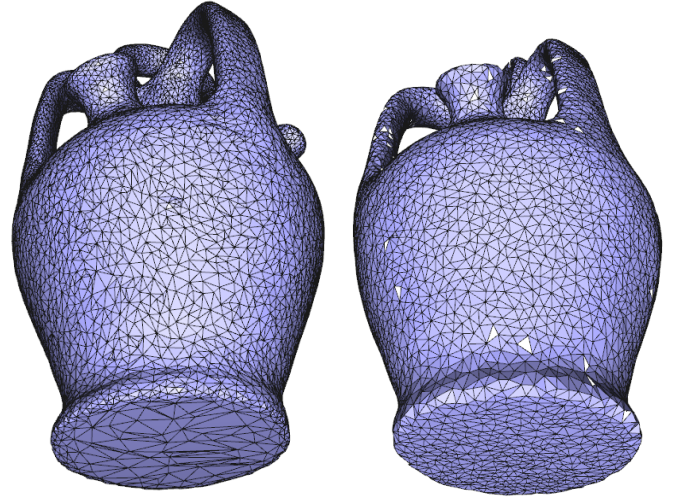


Figure 10: The left image is the original mesh with 14859 vertices. There are a lot of large cells at the bottom of the vase visible and some very small cells along the side of the vase. On the right, the mesh recreated from the 10000 Poisson disk sampling is shown, the cells seem to have a more uniform size.

The cell area is shown in a histogram in the original mesh and of the reconstructed mesh according to the point cloud.

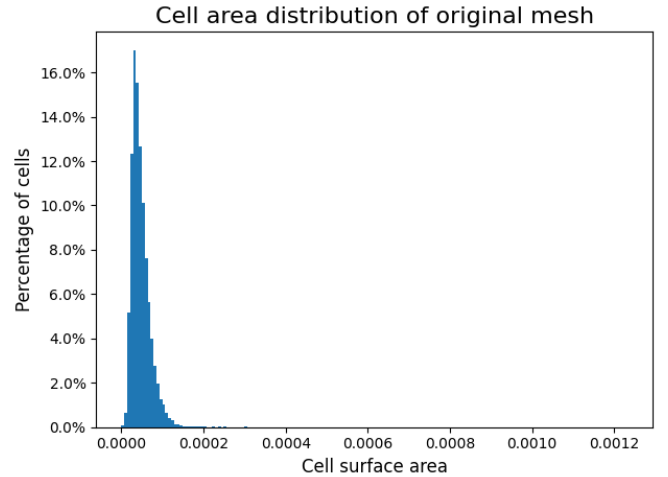


Figure 11: The distribution of the cells in the original mesh go from near 0 to approximately 0.0012 (the bar is too small to show in the image). These large cells are most likely the cells on the bottom of the vase mesh. Note also cells with very low surface area are visible in the plot.

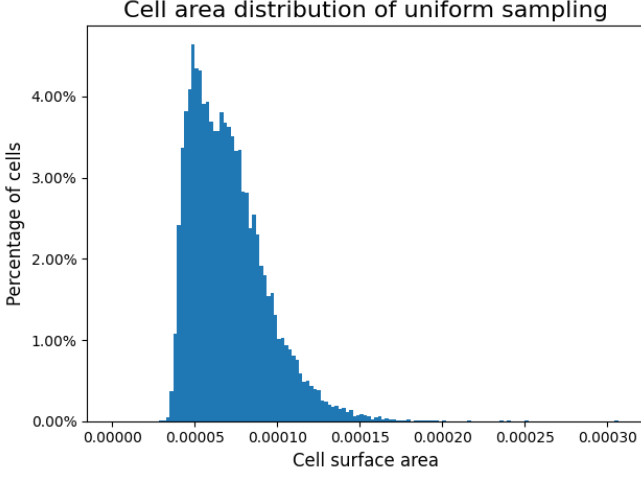


Figure 12: The distribution of the cells after the surface area after surface reconstruction. The maximum cell area is less of an outlier compared to the original situation. The minimum starts around 0.00004 instead of 0.

Since resampling is especially important for the alignment step later a mesh with very few vertices will be briefly discussed as well using [Figure 13](#) and [Figure 14](#).

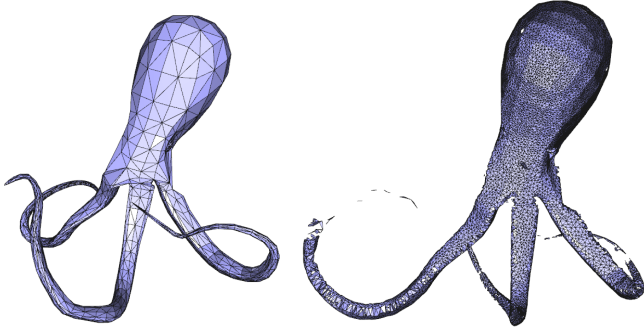


Figure 13: The left image is the original mesh with 1343 vertices. Besides the low number of vertices there is also a large difference in cell size. On the right, the mesh recreated from the 10000 Poisson disk sampling is shown, the cells seem to have a more uniform size. Note the right mesh is only a temporary representation of the point cloud, so the inverted faces are not present in the used mesh.

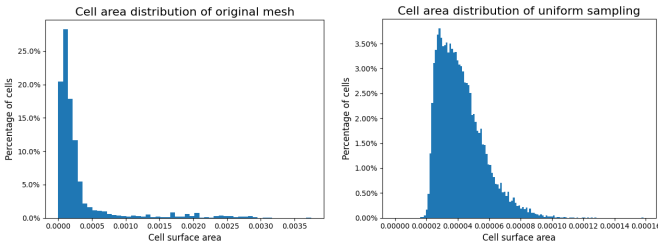


Figure 14: The distribution of cell surface area of the original mesh and the Poisson disk sampled point cloud mesh. The minimum cell area does not start at around 0, and the maximum surface area is closer to the main range of surface area in the mesh.

Using this uniformly representation of the mesh, the point vertices can be used to perform the normalization steps.

3.6. Step 2.5: Normalizing shapes

In this section, both the translation and scaling steps of the normalization pipeline are explained. Plots are provided to show the distribution of the features before and after normalization. These normalizations are done since, for example, the location and the size of an object do not matter to whether two shapes are similar. From the user's perspective, these normalizations only change the viewpoint and distance relative to the object.

3.6.1. Translation

The mesh is translated to the origin so, the subsequent steps do not need to adjust for the current position of the mesh. The center of a mesh is computed by taking the barycenter. This is computed by taking the average coordinate of all the vertices. To determine the actual barycenter, Open3D is used.

$$\text{barycenter}(M) = \frac{\sum_{j=0}^m v_j}{m} \quad (1)$$

Equation 1: Equation to compute the barycenter of a mesh. The summation and division are applied to the components of v_j . M = mesh, m = number of vertices, v_j = vertex at index j

The mesh is then translated by subtracting the barycenter x, y, and z components from the vertices of the mesh. The equation is shown below.

$$\text{translated}(\vec{v}_j) = \vec{v}_j - \vec{c} \quad (2)$$

Equation 2: Translation step of a single vertex using the barycenter. v_j = vertex j of the mesh, c = $\text{barycenter}(M)$

To verify the translation has been done correctly, the distance to the origin is compared. After translation distance from the barycenter to the origin is expected to be 0. To do the actual translation, Open3D is used.

$$d(\vec{c}, O) = \sqrt{\vec{c} \cdot \vec{c}} \quad (3)$$

Equation 3: Equation to compute the distance of c to the origin. O = origin, c = barycenter of M

These are the results of the translation normalization step ([Figure 15](#) and [Figure 16](#)).

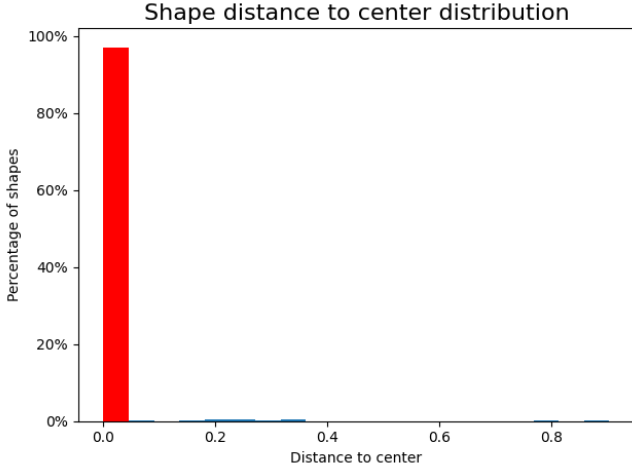


Figure 15: The distance to the center for most shapes is slightly over 0. However, there are some shapes that are more than 0.1 away from the center.

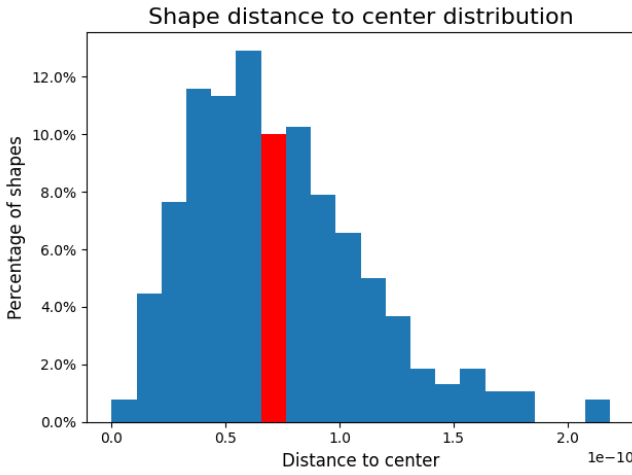


Figure 16: The distance from the center after normalization is much closer to 0. As seen in the bottom right corner the distance from the center is a maximum of approximately 2.2×10^{-10} . This is due to rounding errors of the vertex coordinates.

All shapes are very close to the origin of the coordinate system after the translation normalization step.

3.6.2. Scaling

The mesh is scaled to an AABB of unit size. The box has 3 dimensions. The extent of the AABB of the mesh by finding the largest dimension and computing the scalar from that.

$$\sigma(P) = \max(x_{\max} - x_{\min}, y_{\max} - y_{\min}, z_{\max} - z_{\min}) \quad (4)$$

Equation 4: Where $x_{\max}$ is the maximum x coordinate of all vertices $(0, \dots, m)$ and $x_{\min}$ is the minimum x coordinate of all vertices in point cloud P .

$$\text{scalar}(P) = \frac{1}{\sigma(P)} \quad (5)$$

Equation 5: Where $\sigma(P)$ is maximum extent of point cloud P .

To normalize the mesh, The vertex coordinates are all divided by the scale. This process is shown for a single vertex in Equation 6.

$$\text{scale}(\vec{v}_i) = \text{scalar}(P) * \vec{v}_i \quad (6)$$

Equation 6: Scales a single vertex using a scalar. P = Point cloud, v_i = vertex i of P .

To verify the correctness of the scaling step. The maximum length of the longest axis of the AABB is expected to be 1. To do the actual scaling, Open3D is used.

These are the results of the scaling normalization step.

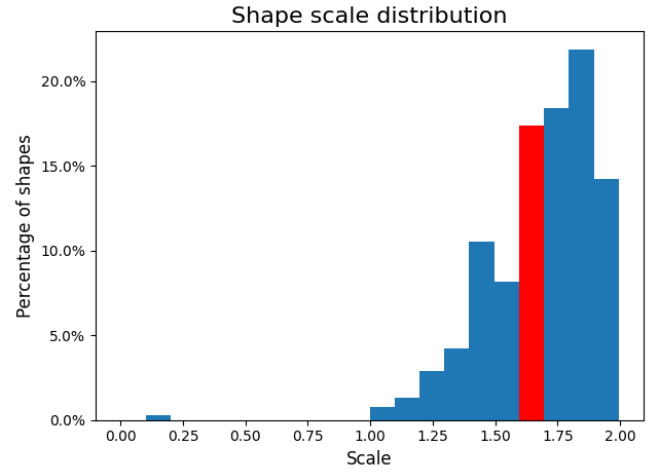


Figure 17: Scale distribution of the meshes in the database before normalization, most of the scales of the meshes are between 1.0 and 2.0.

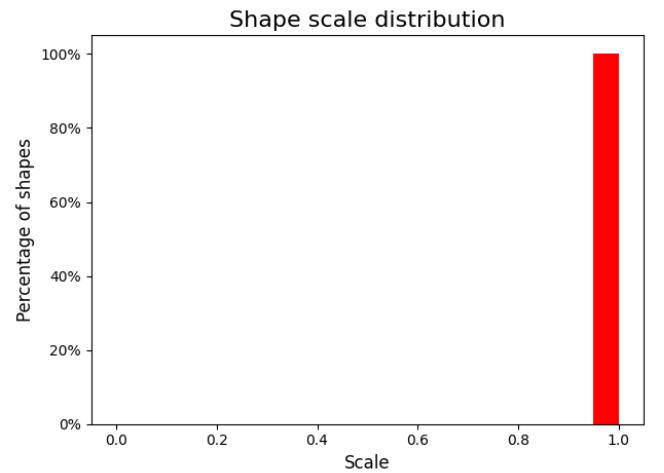


Figure 18: Scale of the mesh after normalization, as expected the scale is now consistent for the entire database. There are small deviations in the data because of float precision

4. Step 3: Feature extraction

In this section, the normalization of the meshes will be finalized and evaluated. Then the methods for feature extractions for both the 3D mesh and the 2D appearance of the mesh are listed and explained. Some of the results will be demonstrated with examples.

4.1. Step 3.1: Full normalization

The final two steps of normalization are explained here. Alignment and flip tests are to normalize the camera angle from which we look at the object.

4.1.1. Alignment

Aligning the shape is done by computing the eigenvectors for the set of vertices $\{v_j\}$. This calculation gives three eigenvalues ($\lambda_1, \lambda_2, \lambda_3$) and three eigenvectors ($\vec{s}_1, \vec{s}_2, \vec{s}_3$). The eigenvalues are the length of each respective vector along which the points are spread. The eigenvectors themselves are normalized.

For normalization, we want the major eigenvector ($\vec{s}_1$) to align with the x-axis. The second eigenvector ($\vec{s}_2$) along the y-axis and the third ($\vec{s}_3$) eigenvector along the z-axis. This is done by sorting the eigenvectors by their eigenvalues.

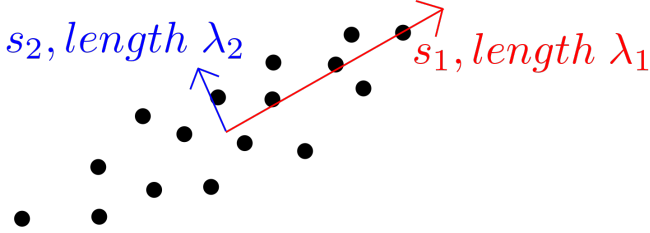


Figure 19: 2D example of the eigenvectors. The eigenvectors follow the angle of the point spread.

A short example of the alignment steps is shown to illustrate the process. Given the following eigenvalues and eigenvectors:

0.064	0.007	0.214	=Eigen values =Eigen vectors
0.996	0.090	0.002	
0.002	0.000	-0.999	
-0.090	-0.996	0.002	

Table 2: Example of eigenvalues and eigenvectors, each eigenvector is a column.

Then the columns are sorted, in descending order, based on the eigenvalues.

0.214	0.064	0.007
0.002	0.996	0.090
-0.999	0.002	0.000
0.002	-0.090	-0.996

Table 3: Eigenvalues and eigenvectors after sorting them in descending order. The eigenvector columns create the rotation matrix that is used to align the mesh.

The (3x3) rotation matrix is constructed from the sorted eigenvectors. If the determinant has a negative

sign, then the mesh is rotated using the minus rotation matrix instead. Otherwise, the cell's normal vectors are oriented inwards. For each vertex, the following transformation is performed.

$$\text{aligned}(\vec{v}_i) = R * \vec{v}_i \quad (7)$$

Equation 7: Where $\vec{v}_i$ is vertex i of the point cloud and R is the rotation matrix

After applying the rotation matrix, re-computing the 3 eigenvectors on P should give a matrix with 1 and -1's at the diagonal and 0 at the other places. Then computing the average absolute dot product between each eigenvector and the component vector is 1.

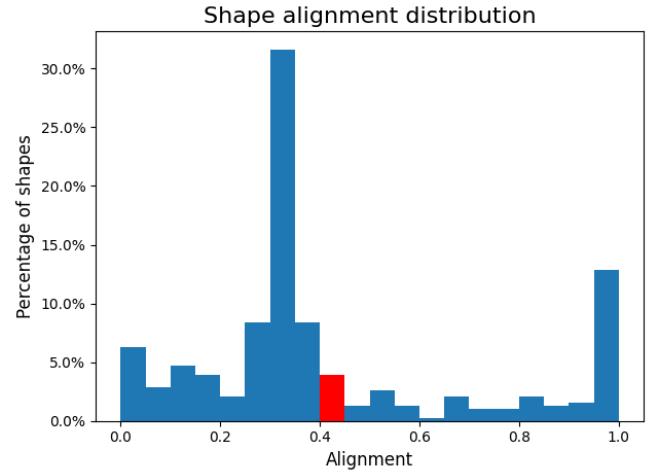


Figure 20: Alignment of the mesh before normalization. The alignment is spread over the entire range of possible values.

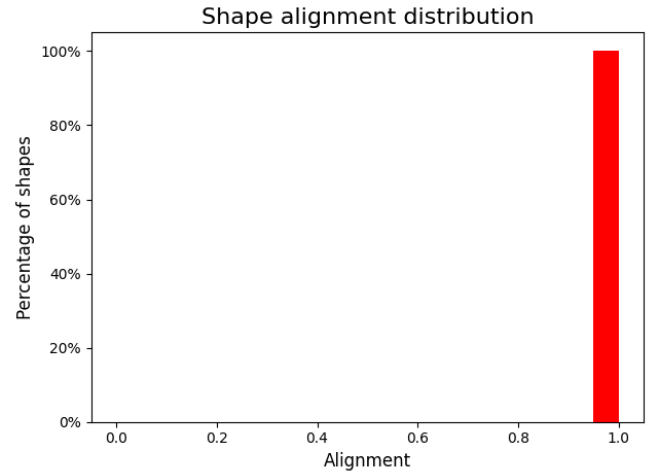


Figure 21: Alignment of the mesh after normalization. As desired all the shapes in the database are perfectly aligned along their eigenvectors.

4.1.2. Flipping

Flipping is done after alignment because the eigenvectors are not oriented. To normalize all types of mirroring of the mesh, the moment test is done.

In order to do the moment test, the center of the triangle needs to get computed. This is done by averaging the x, y, and z coordinates of the vertices of each triangle.

$$C_i = \frac{A + B + C}{3} \quad (8)$$

Equation 8: Where $C_i = i^{th}$ coordinate of center of triangle t ($i \in \{x, y, z\}$). A, B, and C are the vertices of the triangle

Then f_i , the mass for each of the axes is summed using the following formula.

$$f_i = \sum_t \text{sign}(C_{t,i})(C_{t,i})^2 \quad (9)$$

Equation 9: Where $C_{t,i} = i^{th}$ coordinate of center of cell c ($i \in \{x, y, z\}$)

When the f_i is created, now the mesh actually needs to be flipped, so there is always more mass on the left side (negative side) of that axis. The following rotation matrix is used.

$$F = \begin{bmatrix} \text{sign}(f_x) & 0 & 0 \\ 0 & \text{sign}(f_y) & 0 \\ 0 & 0 & \text{sign}(f_z) \end{bmatrix} \quad (10)$$

Equation 10: Rotation matrix used to mirror the mesh.

When the determinant of the matrix F is negative, the backsides of the triangles are shown, which indicates the normals are incorrect. To resolve this, the order of points in the triangles is reversed.

The amount of positive elements in f_i indicates how many axes were already correct using the momentum test. After normalization, it is expected that all 3 axes are correct based on the moment test.

$$flip = \sum_{i=0}^2 \text{sign}(f_i) \quad (11)$$

Equation 11: Equation to check how many of the axes have the correct orientation, where most of the mass is on the left side of the axis.

Here, the results are shown before and after normalization. The results can only be round values from 0 to 3.

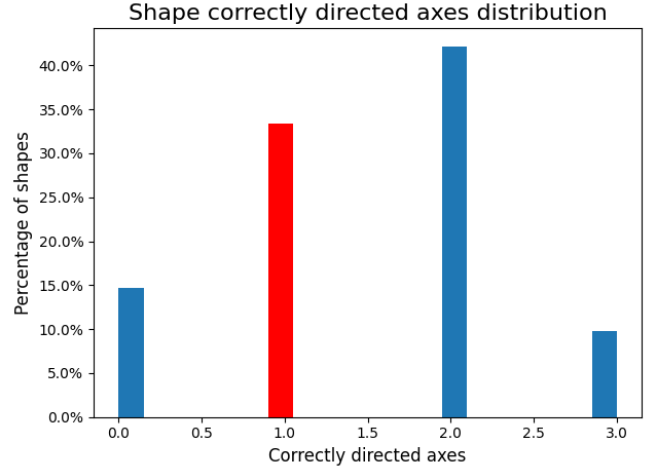


Figure 22: The majority of the meshes still need to be flipped along an axis.

After normalization, the desired result is reached. All the meshes are flipped correctly and have all 3 axes satisfy the moment test (Figure 23).

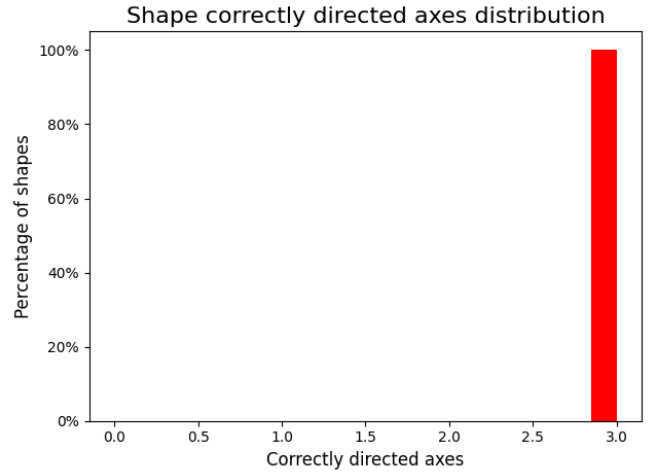


Figure 23: Number of axis that are correctly flipped. Where most of the weight is on the left side of that axis.

4.2. Step 3.2: 3D shape descriptors

Write about why we have these descriptors. In short, to query for similar shapes later These descriptors are combined later in a feature vector.

4.2.1. Surface area

The surface area of a single cell (c_i) is given by the following formula:

$$surfaceArea(c) = \frac{|\vec{AB} \times \vec{AC}|}{2} \quad (12)$$

Equation 12: c = cell, and A, B, C are the three respective vertices of the cell

The surface area (S) of a mesh is the sum of all the individual cell areas, as shown in equation 13.

$$S(M) = \sum_{i=0}^n \text{surfaceArea}(c_i) \quad (13)$$

Equation 13: Sum of all the cell surfaces that define the mesh. c_i = cell i of mesh M

A short showcase of shapes with different perceived surface areas is shown below.

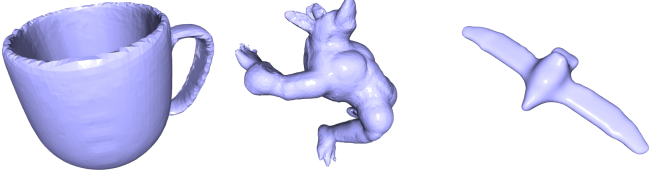


Figure 24: A few examples of meshes with different surface areas. The surface area decreases from left to right. Their surface areas are respectively 3.65, 1.45, and 0.35.

4.2.2. Volume

This is a feature that will be used in combination with surface area to compute how compact a mesh is. The volume of a mesh is computed by creating a tetrahedron formed by the 3 vertices of the triangle and the origin.

$$\text{cellVolume}(c_i) = \frac{(A \times B) * C}{6} \quad (14)$$

Equation 14: Where c_j = cell j of the mesh. A, B, and C are the three respective vertices of the cell

To get the total volume of the mesh all the volumes of the cells are added together using the following formula.

$$V(M) = \left| \sum_{i=0}^m \text{cellVolume}(c_i) \right| \quad (15)$$

Equation 15: Where c_i = cell i of the mesh. A, B, and C are the three respective vertices of the cell

To give an explanation of how this can compute the volume of a 3D mesh even if there are non-convex features we have a 2D example. Note that the points do not have to line up for it to give the correct volume.

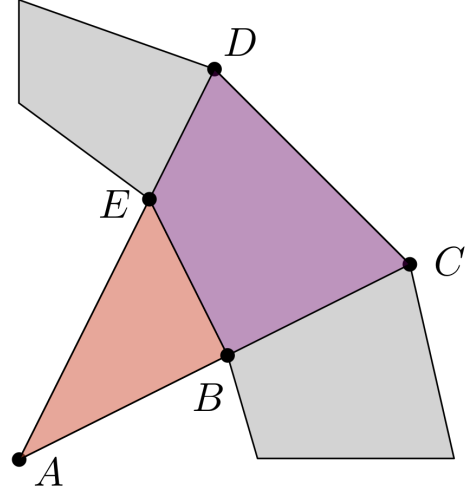


Figure 25: To compute the area (volume in 3D) of BCDE using the edges (cells in 3D) CD and BE we can compute the area of ABE and subtract it from the area of triangle ACD. The orientation of the triangle can result in a negative (subtract the volume) or positive (add the volume) volume of the tetrahedron.

To make this method of computing the volume work all cells require a consistent orientation. For that, a recursive method can be used which starts from an arbitrary single cell and orients all cells around it to its own orientation. This then is done for those neighbour cells until all cells are oriented in the same direction. It does not matter if all the cells of the mesh are facing the wrong direction because, in the end, the absolute is taken.

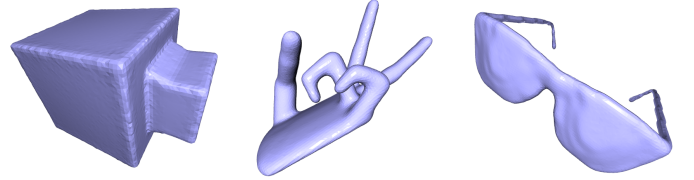


Figure 26: A few examples of meshes with different volumes. The volume decreases from left to right. Their volumes are respectively 0.44, 0.023, and 0.0077.

4.2.3. Compactness

Compactness is defined as how much greater the volume is than the volume of a sphere with the given surface area. This is because a sphere has the minimum surface area for its volume. If the mesh is a sphere, the compactness is 1. The less compact meshes get a higher compactness score. The equation is given below.

$$c = \frac{S^3}{36\pi V^2} \quad (16)$$

Equation 16: Where S = Surface area, V = Volume

A few examples are shown of meshes with different compactness values. One of the most compact shapes of the database is a round vase which is, of course, close to a sphere. As can be seen in Figure 27.

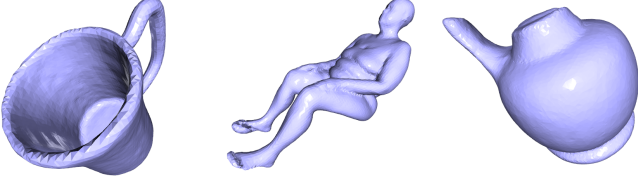


Figure 27: A few examples of meshes with different compactness. Compactness decreases from left to right. Their compactness values are respectively 63.64, 5.01, and 1.48.

4.2.4. Rectangularity

The rectangularity of a mesh is calculated by taking the volume of the mesh and dividing it by the volume of the oriented bounding box (OBB). This results in what fraction of the OBB the mesh fills. After doing alignment using PCA the OBB is equal to the AABB. So its volume can easily be computed using the following formula.

$$V_{OBB} = (x_{max} - x_{min}) * (y_{max} - y_{min}) * (z_{max} - z_{min}) \quad (17)$$

Equation 17: Volume computation of the bounding box where x_{min} and x_{max} are the respective minimum and maximum x coordinate of the vertices that make up the mesh.

If the object is completely rectangular the value will be 1. The less rectangular the closer the value will be towards 0. The rectangularity is computed in the following way.

$$rectangularity(M) = V(M)/V_{OBB} \quad (18)$$

Equation 18: Compute the rectangularity of mesh M . Where $V(M)$ is the volume of the mesh and V_{OBB} is the volume of the OBB.

To show the rectangularity of example meshes the edges of the OBB are included in the images as seen in Figure 28.

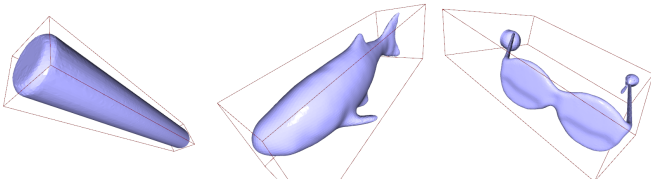


Figure 28: A few examples of meshes with different rectangularity values. The rectangularity decreases from left to right. Their rectangularity feature values are respectively 0.77, 0.23, and 0.01.

4.2.5. Convexity

The convex hull for a 3D shape is intuitively the shape a plastic bag will take when it is perfectly wrapped around the shape. As can also be seen in Figure 29.

Similar to the rectangularity feature, convexity is also added as a more refined measure than rectangularity. It still loses the fine details of the mesh but fits more tightly around the mesh than a box. Convexity describes how much of the convex hull (CH) volume the mesh fills. The

more convex a mesh is, the closer the convexity will increase to 1.

$$convexity(M) = V(M)/V(CH) \quad (19)$$

Equation 19: M = Mesh, CH = Convex hull of M

The Quickhull algorithm[Quickhull3D] can be used to compute the convex hull of a set of vertices in 3 dimensions. In the following image, the convex hull is also shown using red edges.

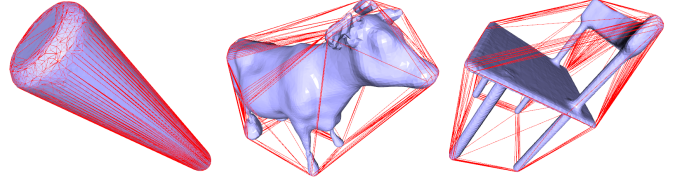


Figure 29: The convexity of the shape decreases from left to right. The respective values are 0.99, 0.42, and 0.07. The bearing on the left almost perfectly fits the convex hull. Whilst the cow takes around 42% of the space.

4.2.6. Diameter

The diameter is defined as the largest Euclidean distance between any two contour points. Because the maximum distance between two points will consist of two points from the convex hull, many candidate vertices can be skipped. This significantly speeds up this feature extraction process.

Since the shape has been normalized to a unit box, the diameter range is in theory 1 to $\sqrt{3}$. However, the database does not contain many shapes above 1.30.

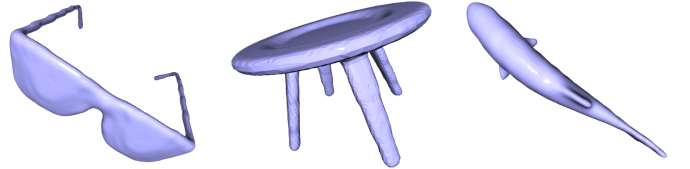


Figure 30: The diameter of the mesh is decreasing diameter from left to right. With their values being 1.31, 1.16 and 0.99. The fish is long but very flat so the longest diameter is between the head and the fins. The glasses' diameter spans from the ear loop to the diagonally opposite end of the glasses. Which makes its diameter significantly longer.

4.2.7. Eccentricity

The eccentricity is the ratio between the amount of spread between the longest and the shortest eigenvector. A sphere has an eccentricity of 1. Since the eigenvalue in each direction will be the same. The more elongated an item is the higher its eccentricity.

$$eccentricity = \frac{|\lambda_1|}{|\lambda_3|} \quad (20)$$

Equation 20: Where λ_1 is the major eigenvector and λ_3 is the minor eigenvector

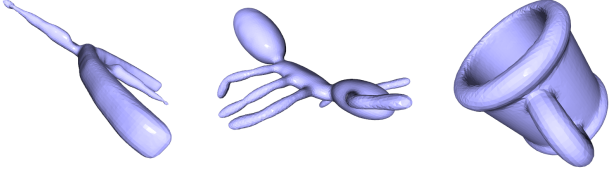


Figure 31: Eccentricity of a few example meshes. On the left, the shape has the most elongation whilst on the right, the points are spread more uniformly over the major and minor axis. Their eccentricity values are 84.53, 7.16, and 1.30.

4.2.8. Major and minor eccentricity

Since eccentricity only looks at the major and minor eigenvalue. There is information lost about the middle eigenvalue. S

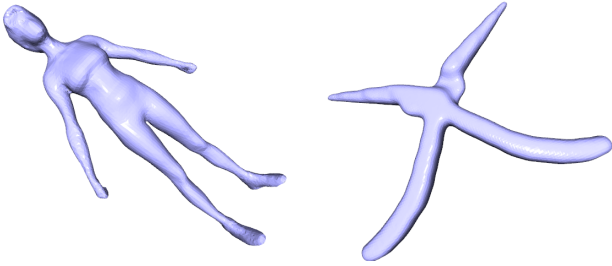


Figure 32: The eccentricity values of these two objects are very similar, 52.82 for the human mesh and 55.50 for the plier. However, the elongation perception is different since the plier is completely 'opened' whilst in the human, the arms are only slightly outward.

To also give an indication of the difference in spread along the first and second eigenvector the second eigenvalue needs to get used in a feature. The following two features are using the information of the second eigenvalue.

$$majorEccentricity = \frac{|\lambda_1|}{|\lambda_2|} \quad (21)$$

Equation 21: Where λ_1 is the major eigenvector and λ_2 is the middle eigenvector

$$minorEccentricity = \frac{|\lambda_2|}{|\lambda_3|} \quad (22)$$

Equation 22: Where λ_2 is the middle eigenvector and λ_3 is the minor eigenvector

Computing these two features separately shows there is a difference between the elongation of both objects since the plier is almost as spread in height as width. So its major eccentricity is 1.22, however, it is very thin so the minor eccentricity gives a value of 45.43. If that gets compared to the human, which has a major eccentricity value of 7.36 and minor eccentricity of 7.17 the difference between the elongation of the two meshes are being reflected well.

4.3. Shape Property Distributions

A shape property function is one that measures various intrinsic geometry properties of a shape. It is invari-

ant to scale, translation, rotation, mesh sampling and pose. Along with this, it needs to be fast and simple to compute and, is specific for different shapes. A shape property distribution is the histogram of such a shape property sampled at a large number of random locations. For each of the properties a different number of samples have been selected. This is because a large fraction of the possible combinations are already measured or because the complexity of the computation doesn't allow for more samples. Each of these histogram features have been distributed over 20 bins of equal size.

4.3.1. A3: Angle between three random points

A3 is a shape property which is the measure of the angle between 3 random points on the surface of an object. The angle is computed in radians and thus has a range of $[0, \pi]$. A total of 8 million angles are computed. The formula used to compute the angle is:

$$A3 = \arccos\left(\frac{\vec{AB} \cdot \vec{CB}}{|\vec{AB}| * |\vec{CB}|}\right) \quad (23)$$

Equation 23: Compute the radian angle using 3 vertices. $|\vec{AB}|$ is the length of a vector.

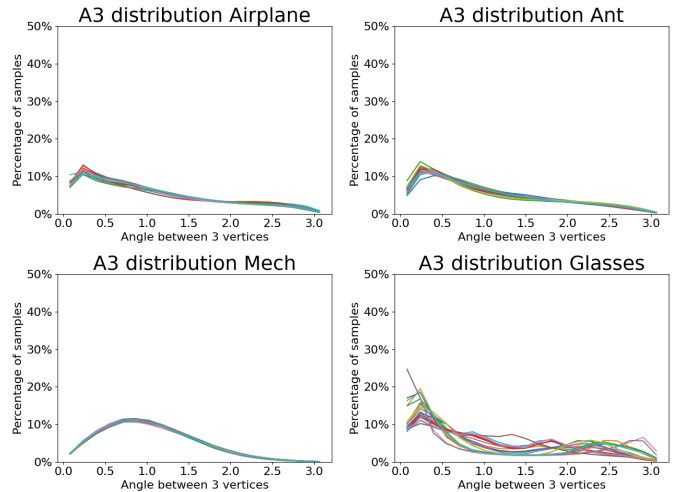


Figure 33: A3 Distribution for 4 different categories. Each category has 20 shapes that have been visualized by a line.

As can be seen in Figure 33, the distribution for airplane, ant are very similar meaning these two categories cannot be well separated using this measure. Mech has a very distinct distribution from the other three classes and is very consistent within the category whilst glasses differ a quite a lot. The complete set of these angles can be found in Figure A.61.

4.3.2. D1: Distance between random point and barycenter

D1 is a shape property which is the measure of the distance between a random point on the surface of an object and the barycenter. A total of 5000 samples are taken for this metric since it is a large fraction of the possible number of samples. The range of this property is $[0, \frac{\sqrt{3}}{2}]$. For completeness the formula is given.

$$D1 = d(A, O) \quad (24)$$

Equation 24: Where d gives the distances between two points. O is the origin (which after normalization is the barycenter), and A is the sampled point.

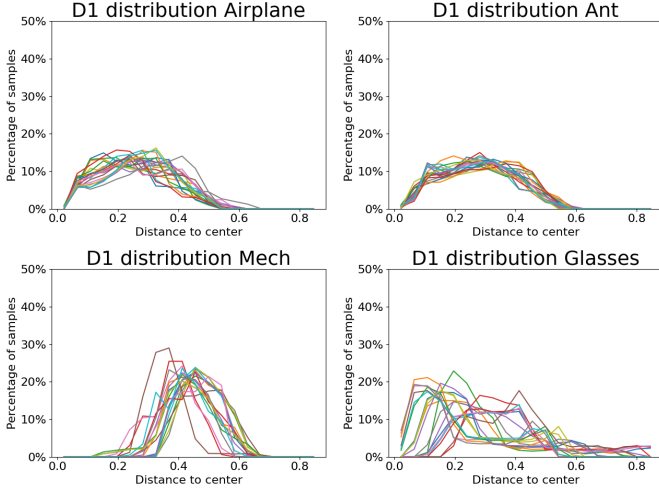


Figure 34: D1 Distribution for 4 different categories. Each category has 20 shapes that have been plotted by a line.

It is immediately clear from Figure 34 this metric is more sensitive than A3 because the lines are not as similar within each class. Whilst all mechs are similar in A3, now they are very different. This indicates that this metric is not a very strong metric to determine similarity between shapes. The distributions for each category is given in Figure A.62.

4.3.3. D2: Distance between two random points

D2 is a shape property which is the measure of the distance between 2 random points on the surface of an shape. A total of 4 million samples have been taken for this property. Since it is normalized to a unit cube the maximum distance is the diameter of the unit cube, which results in a range of $[0, \sqrt{3}]$. The equation is given by Equation 25.

$$D2 = |\vec{AB}| \quad (25)$$

Equation 25: The length of a vector created of 2 sample points $\{A, B\}$

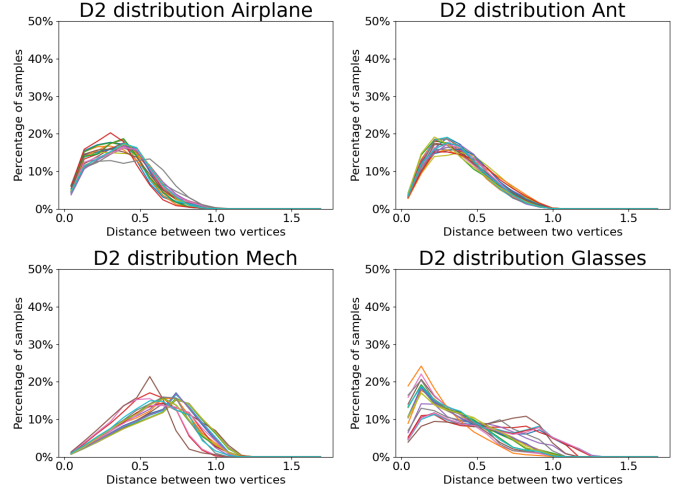


Figure 35: D2 Distribution for 4 different categories. Each category has 20 shapes that have been plotted by a line.

From the plots shown in Figure 37 it can be observed that the range is not entirely used because a lot of shapes do not have an equal spread along all three axis, so there is not a point in both corners of the unit cube that can construct a length of $\sqrt{3}$. Also, each shape has some samples with a length of 1, which is the extend of one axis after normalization. The D2 metric does show clear differences between categories whilst being similar for shapes within the category. Making it a good candidate for matching.

4.3.4. D3: Area of triangle made by three random points

D3 is a shape property which is the measure of the area of the triangle made by joining 3 random points on the surface of an object. The root is taken since it is a 2D metric so it has the same dimensionality. The range of this property is $[0, 1]$. A total of 8 million samples are taken to create the distribution histogram, which is the same number of samples as A3, this is so the sets of points can be used for both properties to speed up computation. The equation is the following:

$$D3 = \sqrt{\frac{|\vec{AB} \times \vec{CB}|}{2}} \quad (26)$$

Equation 26: The square root of the triangle area created by 3 vertices $\{A, B, C\}$

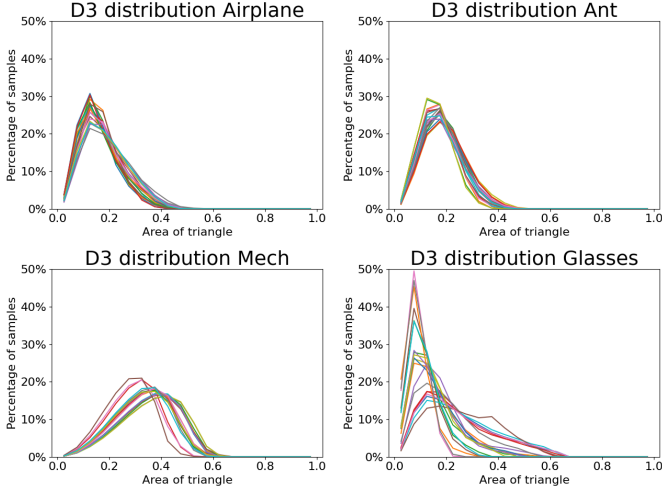


Figure 36: D3 Distribution for 4 different categories. Each category has 20 shapes that have been plotted by a line.

As seen in A3 and the other D metrics discussed so far, the airplane and ant category are quite similar to each other. Also within the category they are similar. The glasses category have their spike at slightly different x coordinates and different heights, but still have similar features that can distinguish them from the mech category. The D3 distributions for all categories can be found in Figure A.64.

4.3.5. D4: Volume of tetrahedron made by four random points

D4 is a shape property which is the measure of the volume of the tetrahedron made by joining 4 random points on the surface of an object. The set of these volumes is cube-rooted and plotted. A total of 6.25 million samples (tetrahedrons) are taken and their volume distribution is plotted per category. The maximum volume for a tetrahedron in a unit cube is $\frac{1}{3}$. So the D4 metric range is $[0, \sqrt[3]{\frac{1}{3}}]$. The equation used to compute the volume of the tetrahedron is given by Equation 27.

$$D4 = \sqrt[3]{\frac{|\vec{AD} \cdot (\vec{BD} \times \vec{CD})|}{6}} \quad (27)$$

Equation 27: The square root of the triangle area created by 3 vertices $\{A, B, C, D\}$

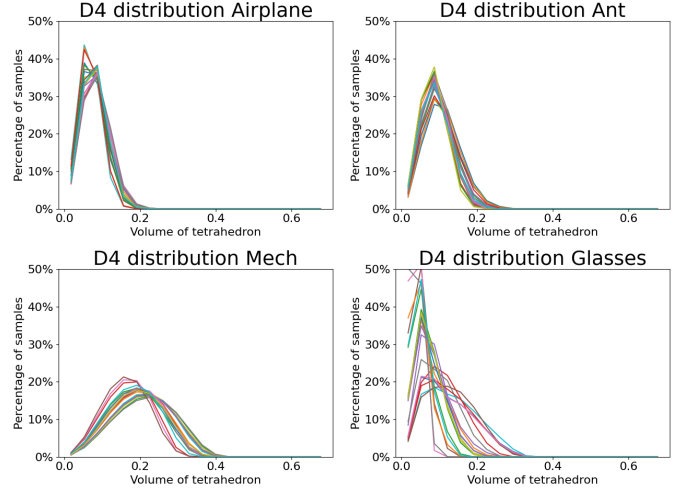


Figure 37: D4 Distribution for 4 different categories. Each category has 20 shapes that have been plotted by a line.

Again there is a clear distinction between the categories, but similar to the other metrics. This indicates how these metrics are relatively good, but do not give a all-round solution to finding similar shapes in the database. This is why all the features will have to be used in combination to give the best results. The full list of D4 distributions for categories can be found in Figure A.65.

5. Step 4: Querying

After extracting the features and saving them to a database it is possible to search through the database for similar shapes to the shape given by the user. First, the shape is normalized like all other shapes in the database before extracting the features. Then the features, stored in a 1D vector, can be compared to other feature vectors in the database.

The dissimilarity of the shapes is measured from 0 to infinity. 0 Meaning the shapes are equal, the higher the dissimilarity value the more dissimilar they are. The first normalization of the features is described after which the dissimilarity measure will be explained.

5.1. Normalizing features

Features described in subsection 4.2 can have different ranges. Some range between 0 and 1, whilst others have a range that can have a very high value. In Figure 38 an example is shown of a feature value distribution in the database.

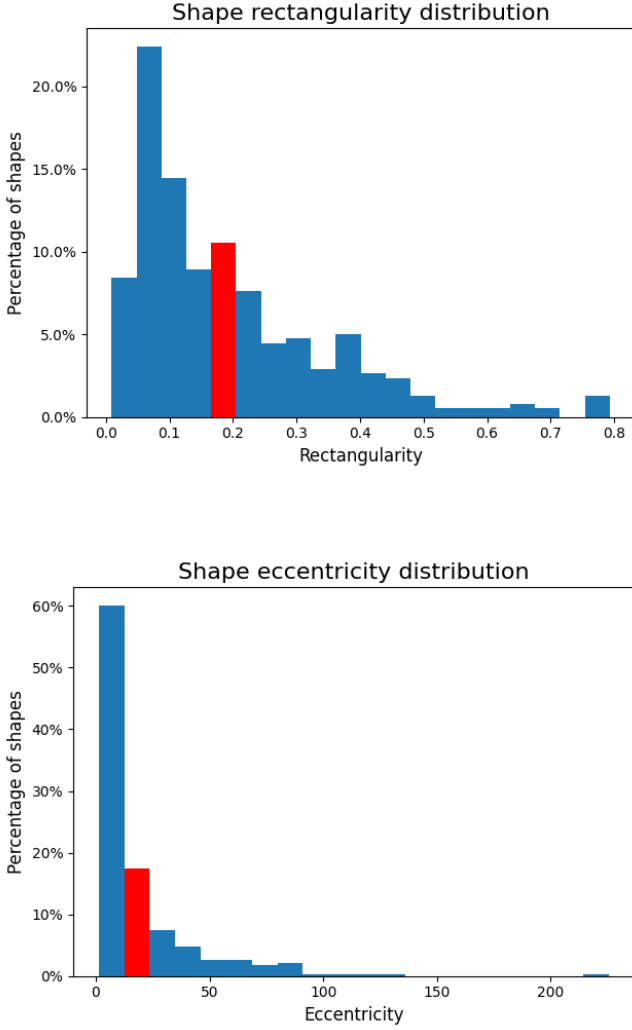


Figure 38: Whilst the rectangularity of the shapes ranges from 0 to 1, eccentricity is spread differently with some having a value higher than 100.

Besides resolving the differences in ranges. Normalizing the feature values also resolves another issue that could occur when querying takes place. If the range of the entire database is between 0 and 100, a shape could be supplied by the user with a hypothetical value of 2000. If the query system could only find shapes within the known range then that query would get rejected.

Normalization takes the mean feature value as 0. Then using the standard deviation assigns all other values to how many standard deviations the value differs from the average value.

Standard deviation is computed using the following formulas.

$$x = |a - \text{mean}(a)|^2 \quad (28)$$

Equation 28: Create a list of squared differences between value and mean. Where a is the list of feature values.

$$\text{std}(x) = \sqrt{\text{mean}(x)} \quad (29)$$

Equation 29: Where x is a list of the squared difference between the value and the mean.

After normalization is done for the two values their distribution is more similar, as shown in Figure 39.

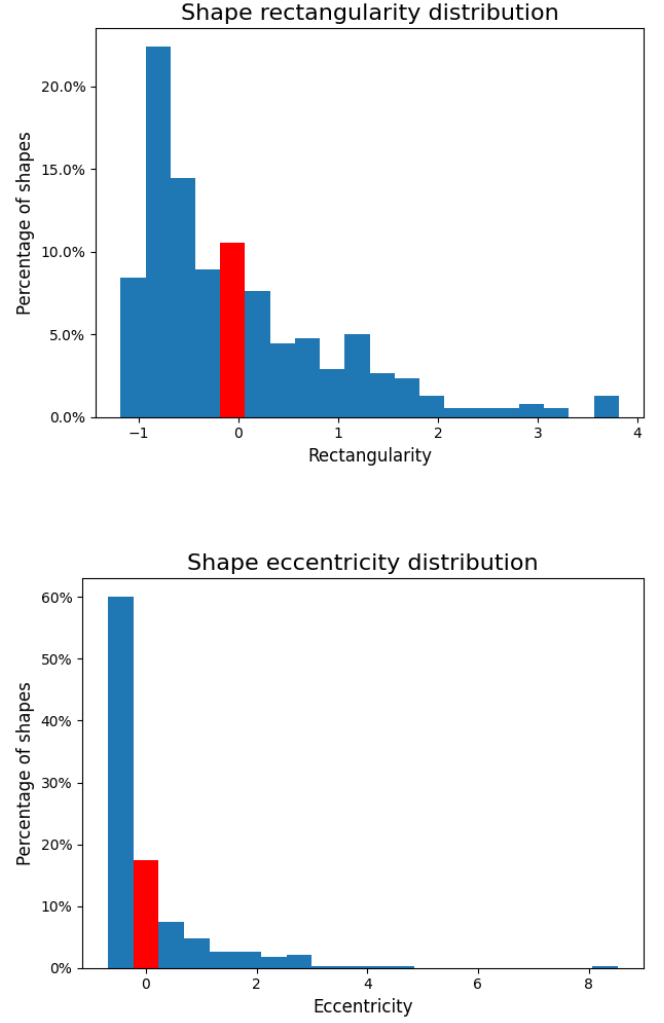


Figure 39: After normalization of the features the average value is 0 as indicated by the red bar in the histograms. The ranges of the feature values are more similar.

The histograms also get normalized by dividing the samples by the number of samples taken. So the area under the histogram becomes 1. Each bin is then the fraction of samples taken that have the given value for that bin in the histogram. As can be seen in the following figure. This translation step had already been done immediately at subsection 4.3.

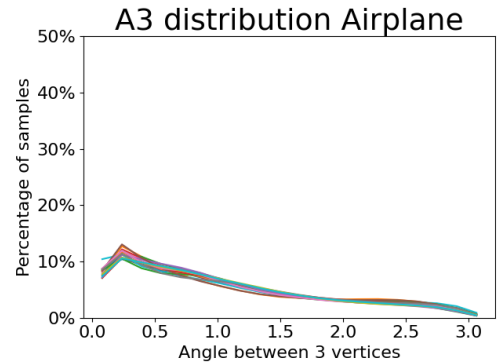


Figure 40: After normalization the feature values total area is one since the percentage of samples within the bin are computed and not the absolute amount.

5.2. Distance metric

After normalization all elementary features have a similar range and all the histogram features have the same area. Now a distance function can be used to measure the dissimilarity between shapes. There are several properties that, if all satisfied, make the distance function a metric. S is the shape collection in the database.

1. Self-identity: $\forall x \in S, d(x, x) = 0$
2. Positivity: $\forall x \neq y \in S, d(x, y) > 0$
3. Symmetry: $\forall x, y \in S, d(x, y) = d(y, x)$
4. Triangle inequality: $\forall x, y, z \in S, d(x, z) \leq d(x, y) + d(y, z)$

For features m number of features that consist of a single value Euclidean distance is used in m dimensions. Euclidean distance satisfies all 4 properties listed above, thus being a metric.

For the histogram features the Earth movers distance (EMD) is used. This stands for how much 'mass' of the bins has to be moved to convert from one histogram to the other. Distance and height are both taken in account. Because the histograms are normalized, each histogram has the same total mass, it is a metric. A modification is done to this metric, making the cost of small movements even cheaper, moving mass over only one bin is $\frac{1}{20}$, the next bin is $\frac{1}{10}$. Increasing the cost multiplication by $\frac{1}{20}$ per bin that needs to be moved. This penalizes small movement less. The total distance for histogram features is the 5 EMD values summed together.

5.3. Feature quality elementary features

What makes a good feature is not only the similarity between similar shapes but also the dissimilarity with shapes of other classes. To inspect the quality of measures, a distance matrix is plotted. The lower the distance the darker the color. The higher the distance, which means higher dissimilarity, gives a brighter color. Assuming shapes of similar classes will have similar feature values, the ideal distance matrix would have dark squares where both the horizontal and vertical shapes are of the same category, but a bright color everywhere else.

5.3.1. Surface area

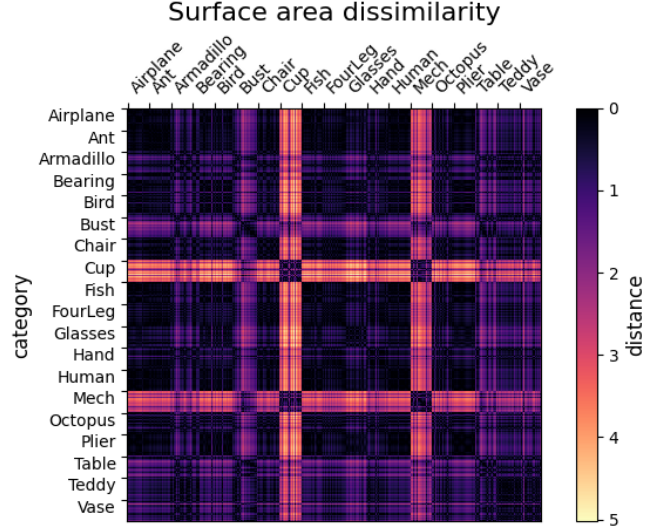


Figure 41: Both the Cup and Mech categories have similar surface areas whilst being very different to the other categories. There is only minimal difference within categories.

As shown in Figure 41, in the diagonal, the distances between shapes of similar categories are low whilst their distances to other classes are in most cases higher. This means the surface area is a good feature.

5.3.2. Compactness

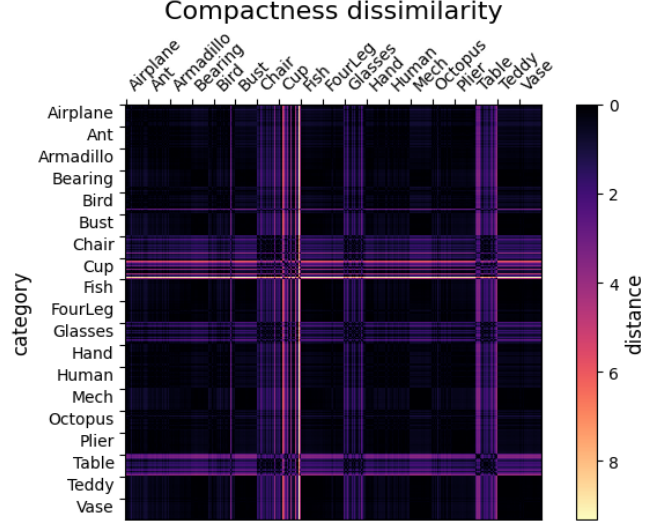


Figure 42: All Vases are very similar in compactness with the Teddies but very dissimilar with tables and chairs because their surface area is very large for their volume.

Only 3 categories have distinct compactness from the other classes. However, they are similar with respect to each other. So the quality of this measure is not very all-round but can maybe distinguish shapes in ways other features cannot. The issues in the cup category are likely due to the errors mentioned in ??.

5.3.3. Rectangularity

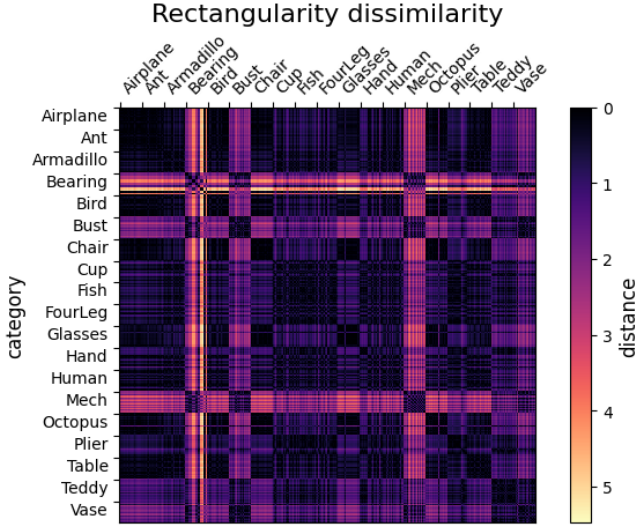


Figure 43: Rectangularity of very cubic elements like busts are very similar. There are however more differences in rectangularity between bearings and cups.

Note that there is a lot of dissimilarity highlighted between different categories and there is a lot of similarity within the categories. So rectangularity is a good distinctive feature.

5.3.4. Convexity

Convexity, which is a refinement of rectangularity, is also visualized to look at the quality of the measure.

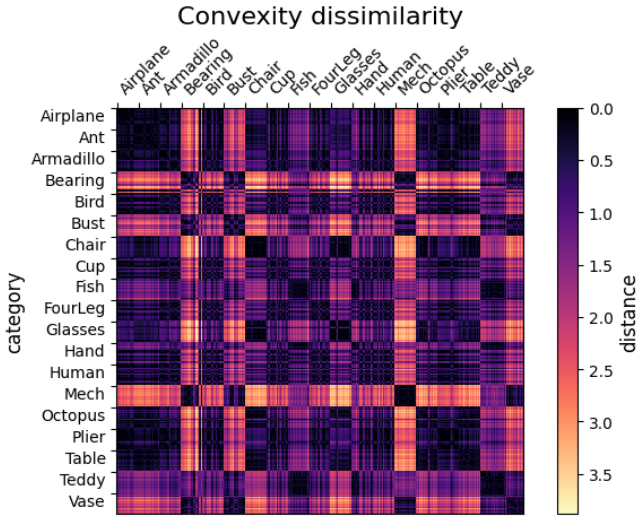


Figure 44: Many of the categories along the diagonal are very similar. Except for cups, almost all of them have a near-perfect similarity. There is also a list of categories for which the dissimilarity is very strong compared to other categories.

It is very interesting to compare the quality of convexity with rectangularity. There are a lot of similarities between the two distance matrices. For example, Bearing, Bust, and Mech are still similar to each other and dissimilar to Airplanes. However, similarity within categories is much clearer, shown by clearer dark areas along the diagonal. Also, there is a lot more dissimilarity between

categories as shown by an increased overall presence of bright colors.

Even though rectangularity is a good feature, this is an overall improvement of that feature. So this feature will be important in determining which shapes that are similar to the query shape.

5.3.5. Diameter

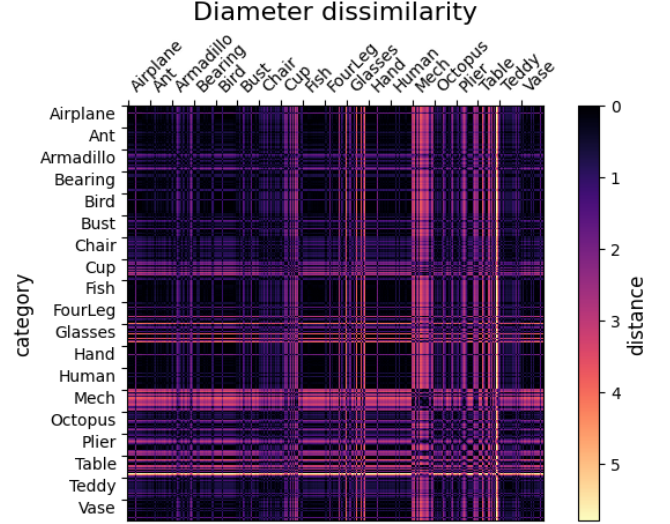


Figure 45: Diameter shows a very distinct dissimilarity matrix. Note there are many 'lines' with dissimilarity even within categories. There are only a couple of categories that have similar diameters.

This measure has a large range compared to the other features so far. Also, this feature is very different from others that have been covered. There are a lot of dissimilarities within categories making it a bad feature.

5.3.6. Eccentricity

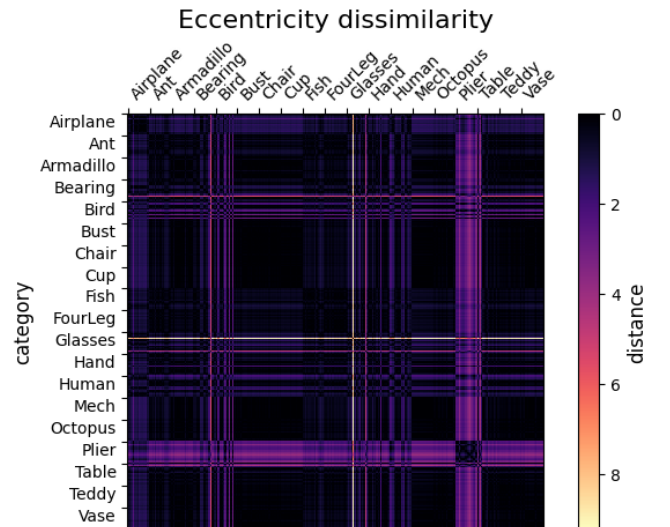


Figure 46: Only the Plier shapes are well differentiated by the eccentricity measure. Those shapes are very wide along one axis and thin along their smallest axis. Other shape categories are mostly similar.

Even though the pliers are very well distinguished using this measure, a couple of other categories are poorly influenced by this feature. Examples of categories for which

the feature is a poor indication are Glasses, Bearings, and Birds. So the quality of this feature is low.

5.3.7. Major and minor eccentricity

As mentioned in subsection 4.2.7 the eccentricity feature is quite limited in showing the perceived elongation of shapes. To see whether the major and minor eccentricities give better indications of their elongation we look at the dissimilarity of those features.

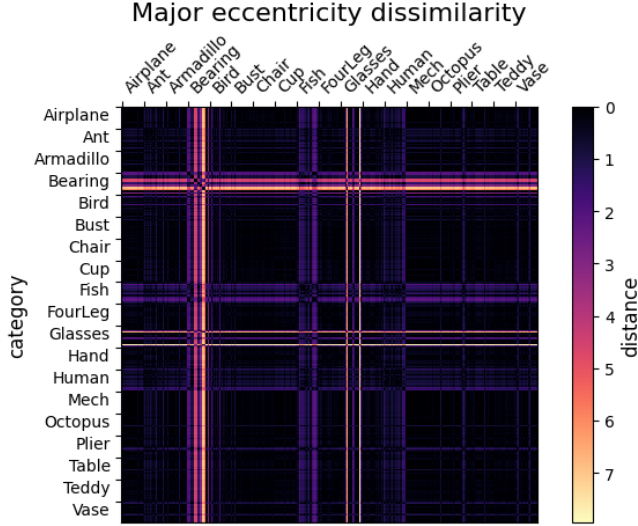


Figure 47: Within bearings the major eccentricity value is different between shapes. However, most of the shapes are very similar to each other.

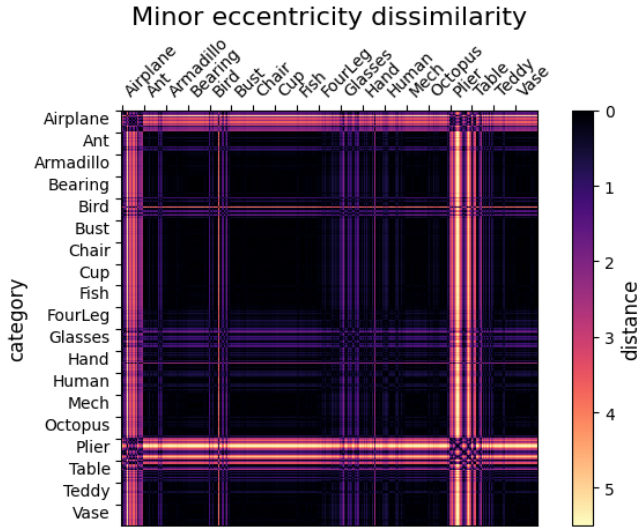


Figure 48: Minor eccentricity gives a strong dissimilarity between airplanes, pliers shapes compared to all other categories. The original eccentricity does not reflect this distinction well. This means that this feature could be useful to differentiate between these types of classes where other features have not been able to distinguish them.

Minor eccentricity gives a strong dissimilarity between airplanes, pliers shapes compared to all other categories. The original eccentricity does not reflect this distinction well. This means that this feature could be useful to differentiate between these types of classes where other features have not been able to distinguish them.

5.4. Feature quality histogram features

Using the EMD feature described in subsection 5.2 distance matrices can be constructed to see their quality.

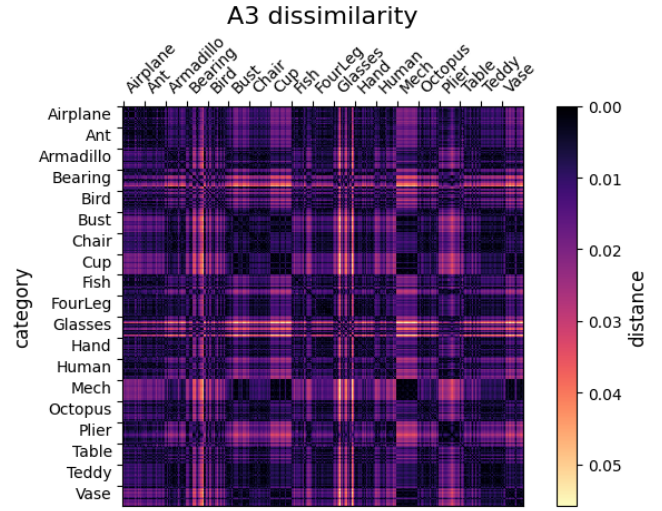


Figure 49: A3 distance matrix for EMD function

The distance matrix for each of the individual D-properties have been computed as well and are shown in Figure 50.

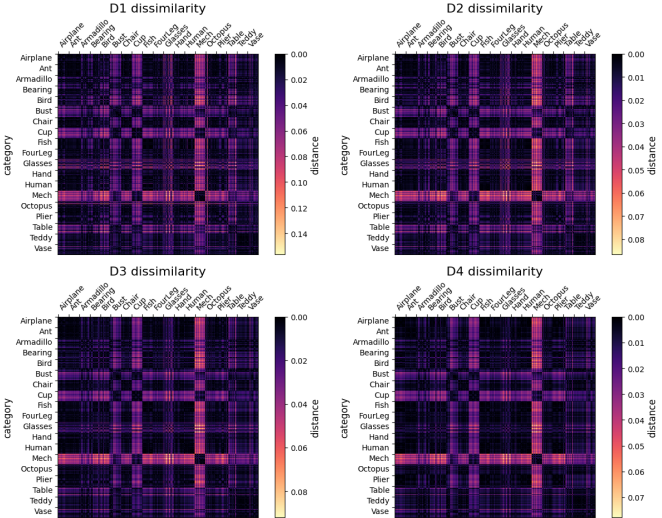


Figure 50: EMD distance matrices for all D metrics for all categories. In general, the matrices look very similar.

As can be seen from all the D-properties distance matrices. All of them highlight similar areas. This means they are all good at finding dissimilarity between the same classes.

5.5. Feature weights

Even though the features are normalized, assigning each of them the same weight would not result in optimal query results. Firstly because the ranges of the features are still different, eccentricity goes to 8 whilst convexity only has a maximum dissimilarity of approximately 3.7. But also because, as described in subsection 5.3, there can be a difference in the quality of the features.

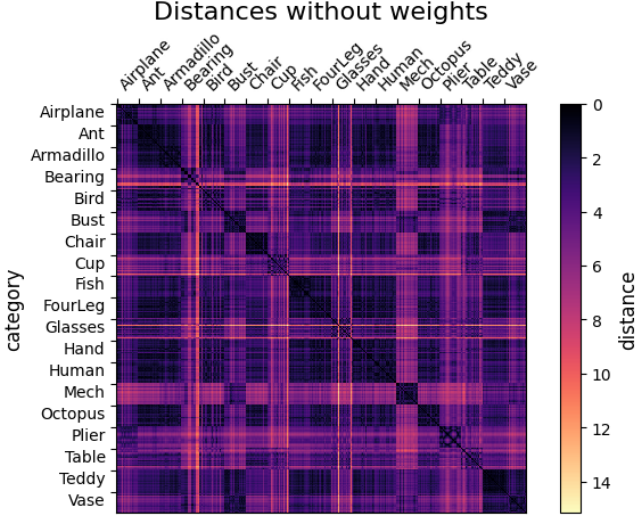


Figure 51: The diagonal rectangles have some internal dissimilarities. But also the dissimilarity with other shapes is not very high.

There are often around 50 other rows in the matrix with a dark enough color which means the dissimilarity with other shapes is low. By giving importance to each feature by a certain weight, it is possible to improve the quality of matches.

5.5.1. Selecting weights

Based on the observed qualities of the features weights were selected by hand. However, it is difficult to estimate the weight of each feature since there are many combinations of weights possible. This is why other approaches were explored. For now we want to optimize the accuracy of the results, wanting as many results of the same category in the query results.

Hyperparameter tuning, which is a method that tries out a lot of combinations of weights, quickly ran into issues with the number of unique configurations of the weight vector. Also, most of the time it would test out combinations that were not promising. A final problem with hyperparameter tuning is that still a rough estimate needed to be created of what reasonable values were.

A more efficient approach, which was then explored and used, looked at promising weight vectors and do small changes to the weights in order to improve the results. Each iteration a random neighbour with the vector weight components individually gets increased or decreased within a random percentage range of $[-20\%, 20\%]$. If this neighbour improves the accuracy it becomes the new optimal weight vector from which new neighbours are created and tested. A larger amount of feature vector weights is adapted after a number of iterations without improvement. This local search approach is used to optimize the accuracy for the search results when returning 10 shapes for each query.

The following weight vector is a result of the described local search method.

Feature	Weight
Surface area	1.17
Compactness	0.30
Sphericity	1.65
Rectangularity	1.07
Convexity	1.14
Diameter	0.14
Eccentricity	0.58
Major eccentricity	0.48
Minor eccentricity	0.13
Histogram feature	
D1	5.01
D2	3.75
D3	2.01
D4	7.64
A3	9.88

Table 4: Overall, the histogram features have a higher weight vector than the elementary features because the range is smaller.

The most important elementary features are sphericity, surface area and convexity. For the histogram features the A3 and D4 are most important. As expected the weight of diameter is low.

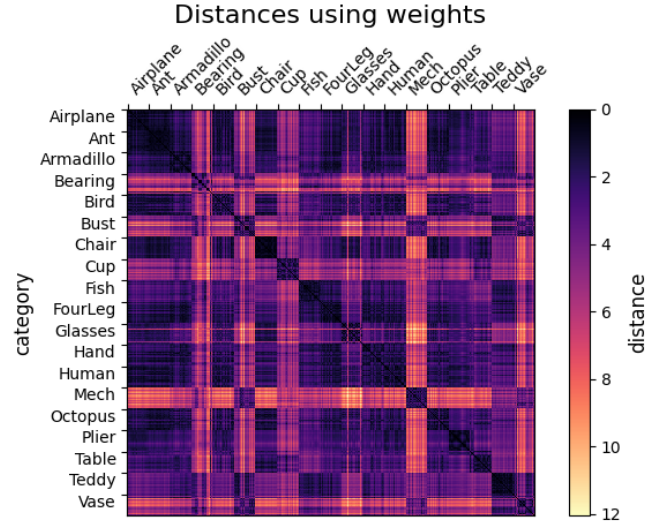


Figure 52: The diagonal has a few internal dissimilarities. However the dissimilarity with other classes is much higher on average.

The resulting distance matrix is shown in Figure 52. There are visible improvements compared to the situation without weights. For example the internal dissimilarity between the plier category is gone. And the cup category as well now is very similar, but dissimilar to other categories. The accuracy got improved from 63% to 68% by tweaking the weights. A confusion matrix will be created and discussed later.

5.6. Querying results

To test the querying results using the described weight vector and EMD distance metric it is able to query the database of normalized shapes. The query shape gets normalized before feature extraction gets performed, then

the first five results which are not the query shape are shown below.

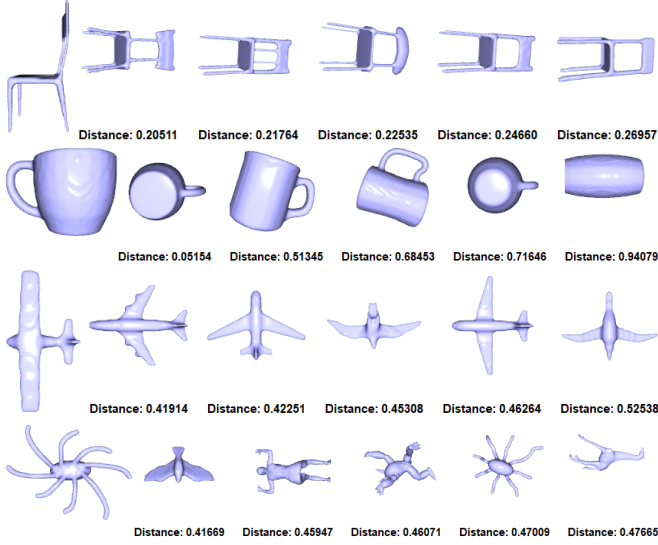


Figure 53: On the left the query shape is shown, then on the right side the first 5 results ordered on their dissimilarity to the query shape. The best query results, the chair, is shown at the top and the worst query results is on the bottom.

In Figure 53 four different query results are shown of shapes in the database. The fifth result in the cup query shows a vase, which is similar to a cup but does not have a handle. The query for a plane also returns two birds. The final query, which is the octopus, returns shapes from many different categories. After inspecting the octopus category it is clear that there are vastly different octopus shapes in the database. Their tentacle count, tentacle direction, and their body size can be vastly different. This can explain why the system is not able to identify other octopus shapes as similar to the query shape.

The distances for the chair shapes have very similar values whilst the cups in general have a higher distance. However, it does result in cups.

6. Step 5: Scalability

The querying approach discussed in section 5 computes the distance between the query shape and all other shapes in the database when a query shape is presented. For the current database size of 380 shape this is feasible, however this does not allow for scaling of the system.

6.1. Step 5.1: Scalable querying

To speed up the query process for an arbitrary amount of shapes in the database we resort to using a high dimensional search algorithm implemented in Pynndescent [Pynndescent]. It creates a neighbour graph where nodes which are close in the search space are connected to each other. When a query shape is presented it starts to search at a random node in the graph until is reached a node close to the query shape. To find the K closest shape it then searches around the node until the k it has likely found the k closest neighbours. The search method has a high performance and accuracy compared to other ANN implementations.

6.1.1. Weights

Since the KNN algorithm constructs the fast query data structure once at the start the EMD metric cannot be used for histograms. This is because the EMD of the query shape can only be computed after the query shape has been presented. Because of this, the histogram distributions will be added to the feature vector. During querying the Euclidian distance is used. Since the metric changed the weights also change, local search has been used again to maximize the number of shapes in the same category as the query shape using $k=10$. It has improved slightly to 71%, meaning that on average 7 out of 10 retrieved shapes from the database are of the same category. The confusion matrix will be discussed in the next section.

Feature	Weight
Surface area	1.07
Compactness	0.50
Sphericity	1.70
Rectangularity	1.35
Convexity	1.82
Diameter	0.08
Eccentricity	0.61
Major eccentricity	0.31
Minor eccentricity	0.12
Histogram feature	
D1	2.73
D2	3.08
D3	0.96
D4	1.71
A3	15.39

Table 5: Weights used for the faster search method using KNN. The elementary weights changed small amounts because their distance metric has not changed. However the histogram features has decrease weights for the D metrics and a large increase, an increase of ≈ 5.51 , compared to the weights of Table 4

The feature vectors of the shapes in the database and the feature vector of the query shape are first multiplied before creating the search data structure and before the query is performed in the data structure. Using the weights is a method to increase the importance of good quality features and reducing the impact of other features that do not provide a lot of information about the category of the shape.

6.1.2. Query quality

The same query shapes of the custom query function are queried again, but now using the KNN query method. The results are then compared. Since the weights do not differ substantially the results are expected to be comparable.

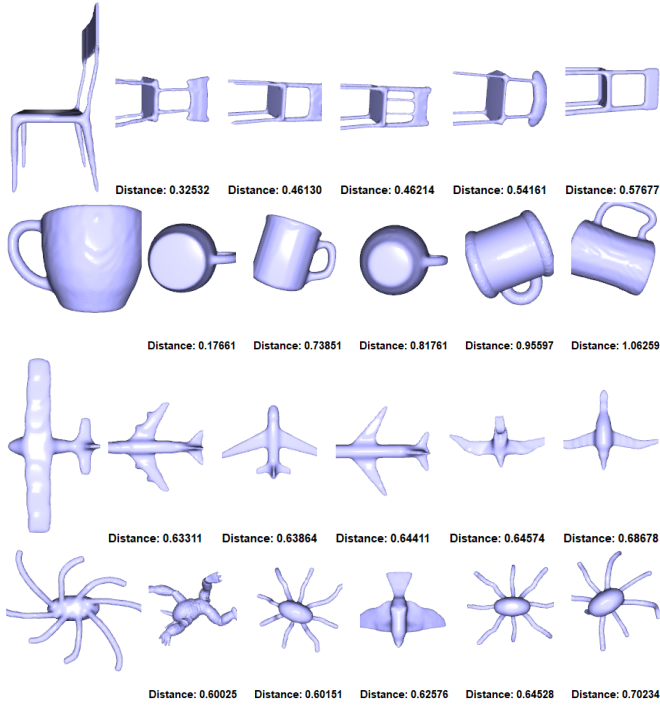


Figure 54: On the left the query shape is shown, then on the right side the first 5 results ordered on their dissimilarity to the query shape. The best query results, the chair, is shown at the top and the worst query result is on the bottom.

For the first 3 queries very similar objects are returned to the custom query implementation. However, the final octopus query now gives 3 results back which are in the same category. This is an improvement over the other implementation. However, conclusions about whether it is substantially better in general will be discussed later.

6.1.3. Query speed

To verify the speed of the query has improved, an benchmark is performed. A query is performed 100 times using a k of 12 on a database of size 380. The average time of a query is then calculated and used to compare the two. The time it takes to extract the features to construct the feature vector is not included in the benchmark time because this is equal for both query methods.

The custom query method takes ≈ 0.62389 second on average per query performed. The KNN query approach takes $\approx 8.203 \times 10^{-4}$ on average which is significantly less.

This benchmark has been performed on a Windows 10 system with the following specifications: Intel(R) Core(TM) i7-7700HQ CPU @ 2.80GHz, 2808 Mhz, 4 Core(s), 8 Logical Processor(s) and 16GB RAM.

6.2. Step 5.2: Dimensionality reduction

Since the feature data are represented in the high dimensional space it is not easy to visually inspect the shape features. The weighted distance matrix in Figure 52 is able to show which categories are similar and which are dissimilar.

Using t-SNE the multidimensional feature vectors can be represented in two dimensions. If two feature vectors are similar in the high dimensional space it will be close to each other in the two dimensional t-SNE representation. A t-SNE representation of the database is shown in Figure 55.

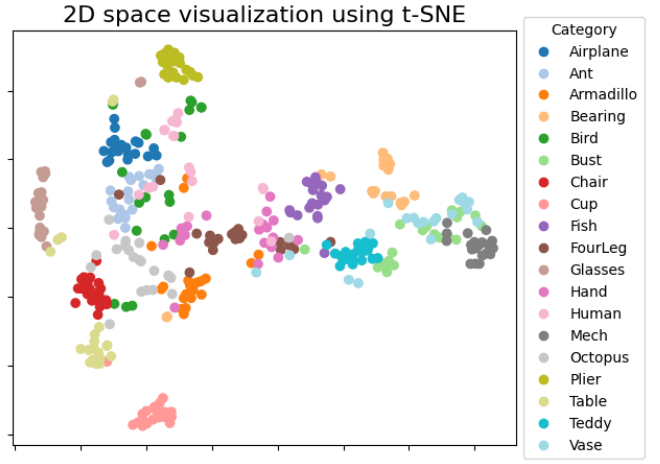


Figure 55: For example: the cup, table, chair, and plier class are very well clustered. These shapes are also all quite similar to each other.

Figure 55 is created using a perplexity of 20. Meaning we expect each shape to have 20 similar shapes. This corresponds to the database where there are 20 shapes of each category. Using the application we can see for example which cup was very similar to the table class in the bottom right.

In the application the name of the file can be inspected when hovering over the point in the t-SNE plot. So it is possible to inspect the cup which is outside of the main cluster.

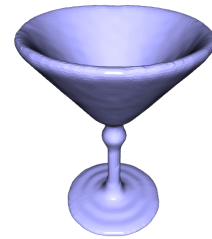


Figure 56: Compared to the cups retrieved in the database (of which a few can be seen in Figure 53) this cocktail glass is very different.

Also, there are two pair of glasses which are located outside their respective cluster and are close to the plier category. These are visualized in Figure 57.

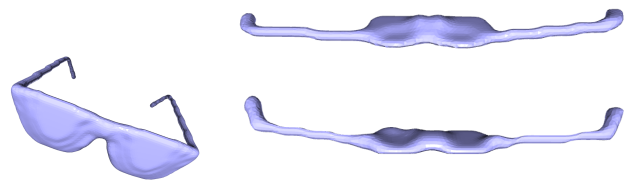


Figure 57: On the left hand side a pair of glasses is shown from the cluster. On the right hand side the two outliers are shown. Both have the temple plate, which is similar to the plier class which are elongated and flat shapes.

7. Step 6: Evaluation

The aim of this section is to evaluate the performance of the shape retrieval system. Each shape is queried and the

results are plotted. We will also show a confusion matrix to evaluate the performance and to identify classes which the system performs well on and which are more difficult.

7.1. Confusion matrix

First the confusion matrix for the custom query method described in section 5. The matrix is normalized over the maximum number of results the confusion matrix can have per category. The values along the diagonal are the true positives.

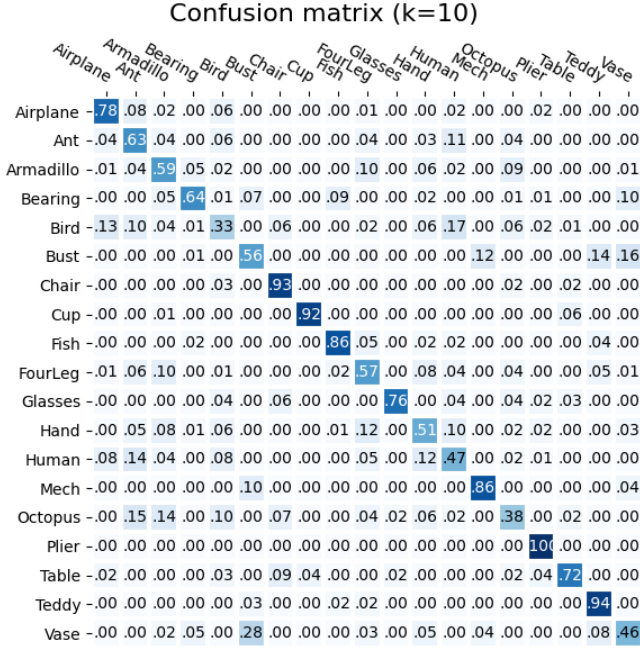


Figure 58: Confusion matrix for k=10 using the custom query method. The true categories are the columns and the predicted classes are listed in the rows.

On average 68% of the query results are of the same category as the query class. As can be seen in the graph, the plier category achieves the perfect score. Meaning it always returns plier shapes when a plier from the database is shown to the system. Chair, cup and teddy also achieve a high score which means these categories are clustered very well together.

Categories the system has more trouble with are for example birds and octopuses. The bird category contains a lot of different types of birds with different wings, which is why this category is more difficult to query consistently. The octopus category, as had been seen in the query results Figure 53, has a lot of different types of shapes which are returned. These categories have in common that they consist out of a body with limbs which are not in line with the main body. Since sphericity is an important metric it is expected that the sphericity of those 3 categories is similar.

The confusion matrix for KNN is also created. The average percentage of returned shape with the same query category increased to 71%. Which is only a slight increase in quality. It is expected that many of the categories that work well for the custom query method also work well for KNN.

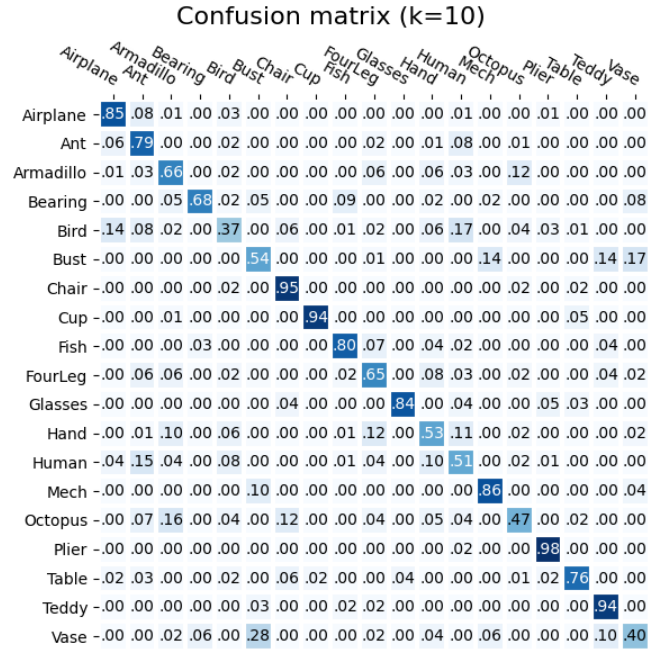


Figure 59: Confusion matrix for k=10 using the KNN query method. The true categories are the columns and the predicted classes are listed in the rows.

The top performing categories stayed the same. There are some significant improvements in the ant category for example, also the octopus category now performs better than before, which corresponds with the query results of that category in Figure 54.

It can also be seen that when querying for vase the retrieval system returns almost as many busts. Both these categories contain few sharp edges, are round and have the similar dimensions to each other.

7.2. Accuracy

We did try to evaluate our program using a different query method, but we were unsuccessful in this regard. We first tried the 'accuracy' measure as defined as followed:

$$accuracy = \frac{(TP + TN)}{|DB|} \quad (30)$$

Equation 30: Where $|DB|$ is the size of the database, TP are the true positives and TN are the true negatives.

In comparison to the previous definition, True Negatives have a much greater influence on the accuracy value since we can only have 20 True Positives maximum while the True Negatives can be 360. True Negatives play the larger role in this definition, but the results did not provide us with any meaningful information (see Table 6).

Category	Accuracy
Airplane	0.96
Ant	0.95
Armadillo	0.94
Bearing	0.94
Bird	0.92
Bust	0.94
Chair	0.98
Cup	0.98
Fish	0.97
FourLeg	0.94
Glasses	0.94
Hand	0.93
Human	0.93
Mech	0.97
Octopus	0.92
Plier	0.98
Table	0.95
Teddy	0.98
Vase	0.93

Table 6: Overall accuracy per class using lecture definition

Unlike the confusion matrix, these values are much closer to each other. This makes it more difficult to spot any big differences between classes. We explored some other metrics to see if they were more fitting, which eventually showed us that there might actually be something wrong with our calculation.

When we tried to create a Relative Operating Characteristic Curve, which uses the sensitivity and specificity metric, we got an unusual graph (Figure 60).

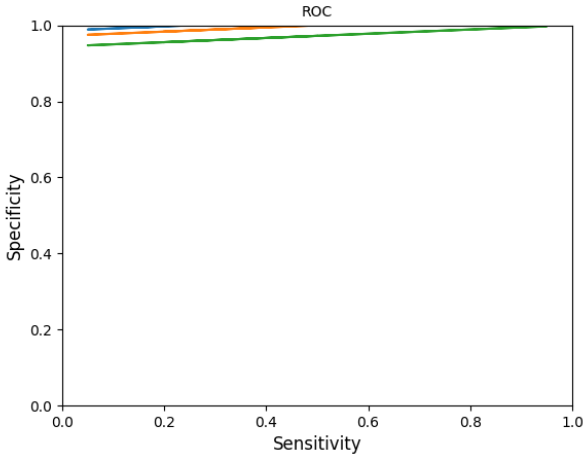


Figure 60: ROC curves with $k = 5$ (blue), $k = 10$ (orange) and $k = 20$ (green)

ROC curves can vary in shape, but they are usually either a diagonal starting in the upper-left corner and ending in the lower-right corner when the system’s performance is at its worst. Or they are a sloped curve that starts and ends in the same corners, but ‘reaches’ towards the upper-right corner in the middle. Yet, our ROC curves are not even close to the worst performance curve.

Time constraints made it so it was not possible to find the issues in the implementation, so we rely mostly on

the confusion matrix to give insights in the performance of the system over the categories.

8. Discussion

The overall system is able to perform reasonably well, the couple of examples shown for both query implementations (Figure 53 and Figure 54) show the system is able to retrieve shapes that look similar to the presented query shape, also for categories that the system performs worse at.

This observation is supported by the 2D space visualization using t-SNE (Figure 55) where it was possible to explore unexpected outliers and support what their dissimilarity was with the rest of that category. Finally, the confusion matrices (Figure 58 and Figure 59) were in line with the observations as well.

8.1. Limitations

Limitations of the system is the speed at which the application functions and the user interactive ability. There are some features missing in the GUI. For example: selecting a custom weight function, selecting K or a maximum dissimilarity. With extra time this can be supported by adding a widget for it.

The more difficult problem is the number of shapes in the current database. The current system only has 380 shapes, making the practical use very minimal. Increasing the number of shapes by a factor 10 will require a lot of extra computation time since Python cannot quickly retrieve the features. Even though optimizations have been performed when possible, the back-end will have to be written in a faster programming language to make the feature extraction fast enough.

Appendix A. Property distributions

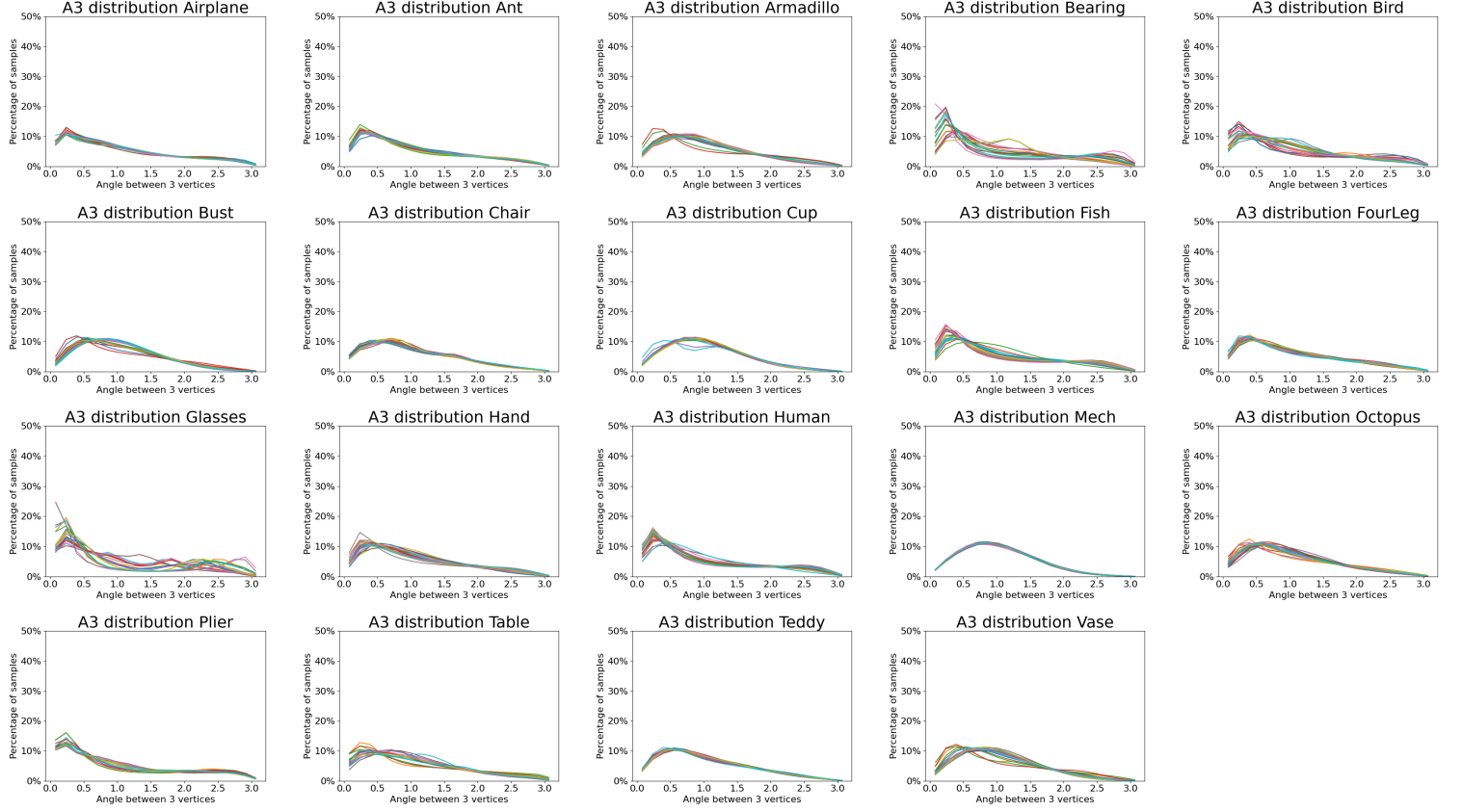


Figure A.61: A3 Distributions for each class in the database

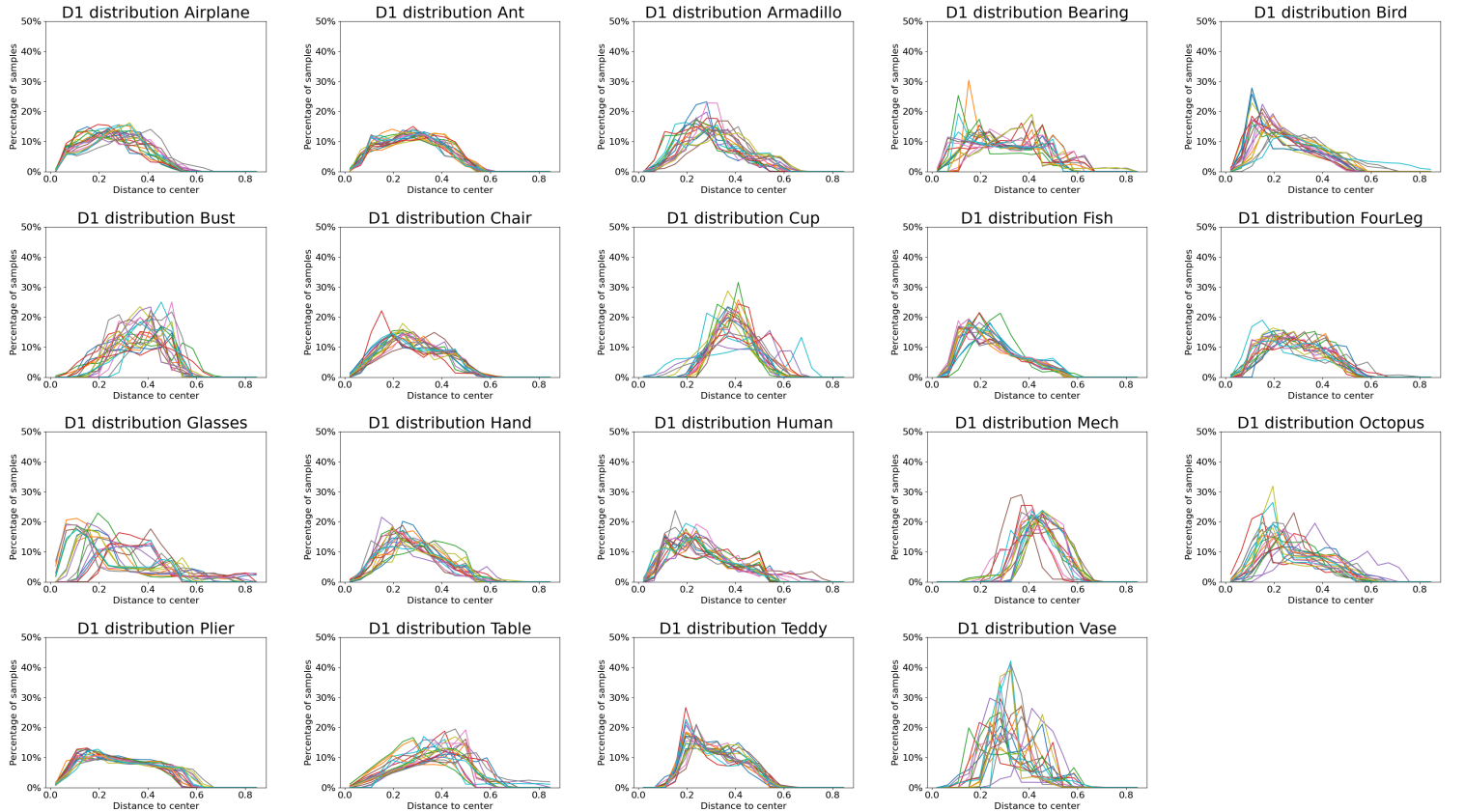


Figure A.62: D1 Distributions for each class in the database.

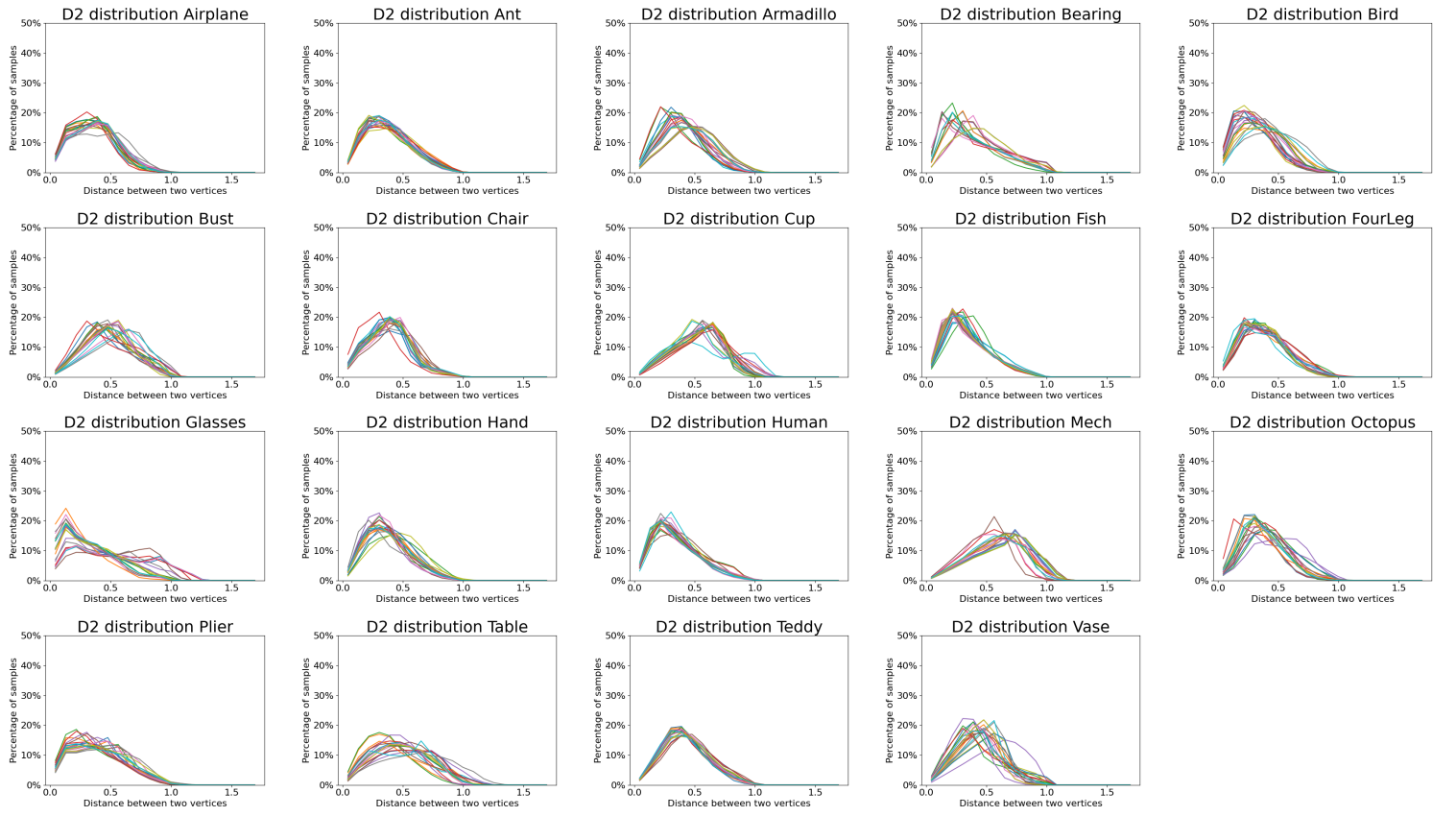


Figure A.63: D2 Distributions for each class in the database

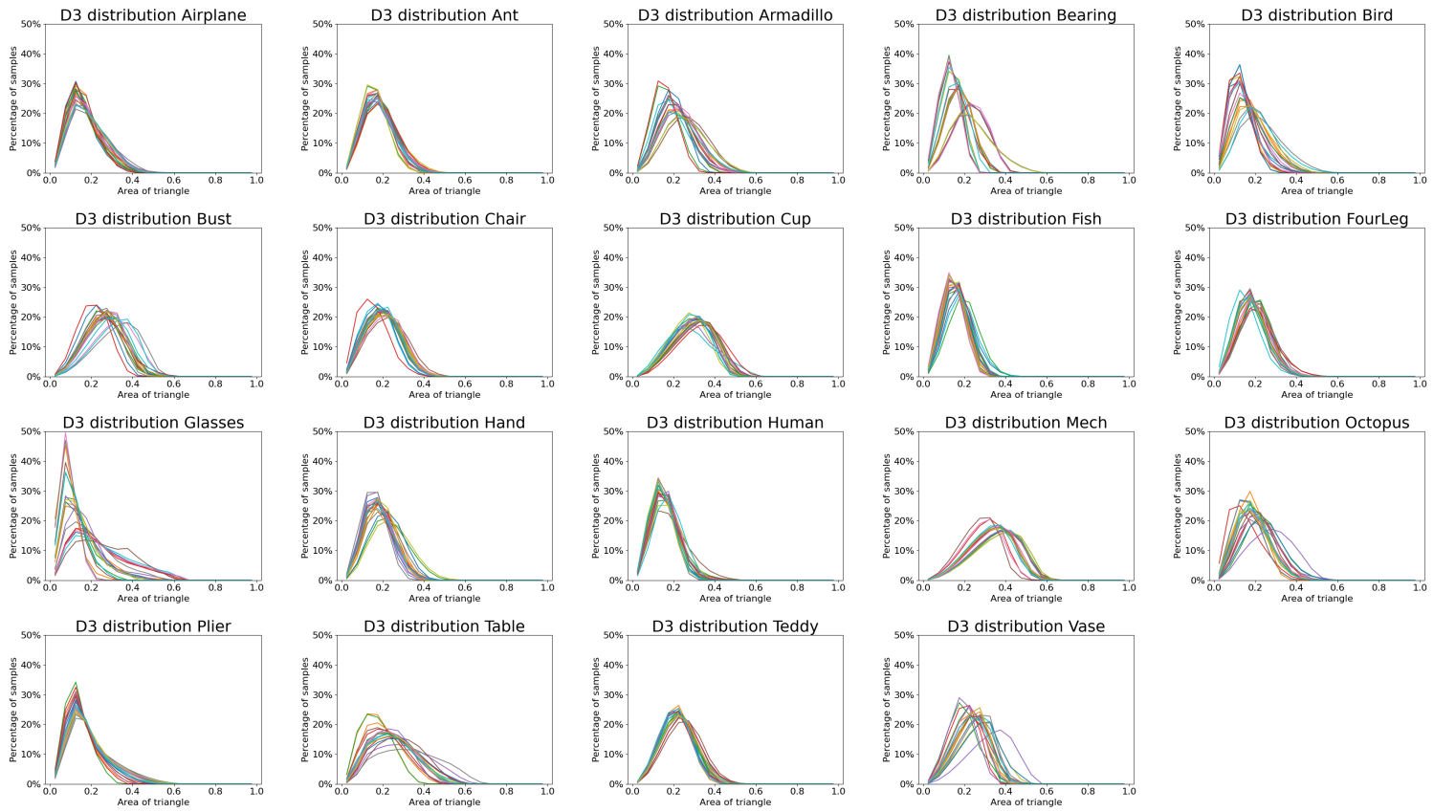


Figure A.64: D3 Distributions for each class in the database

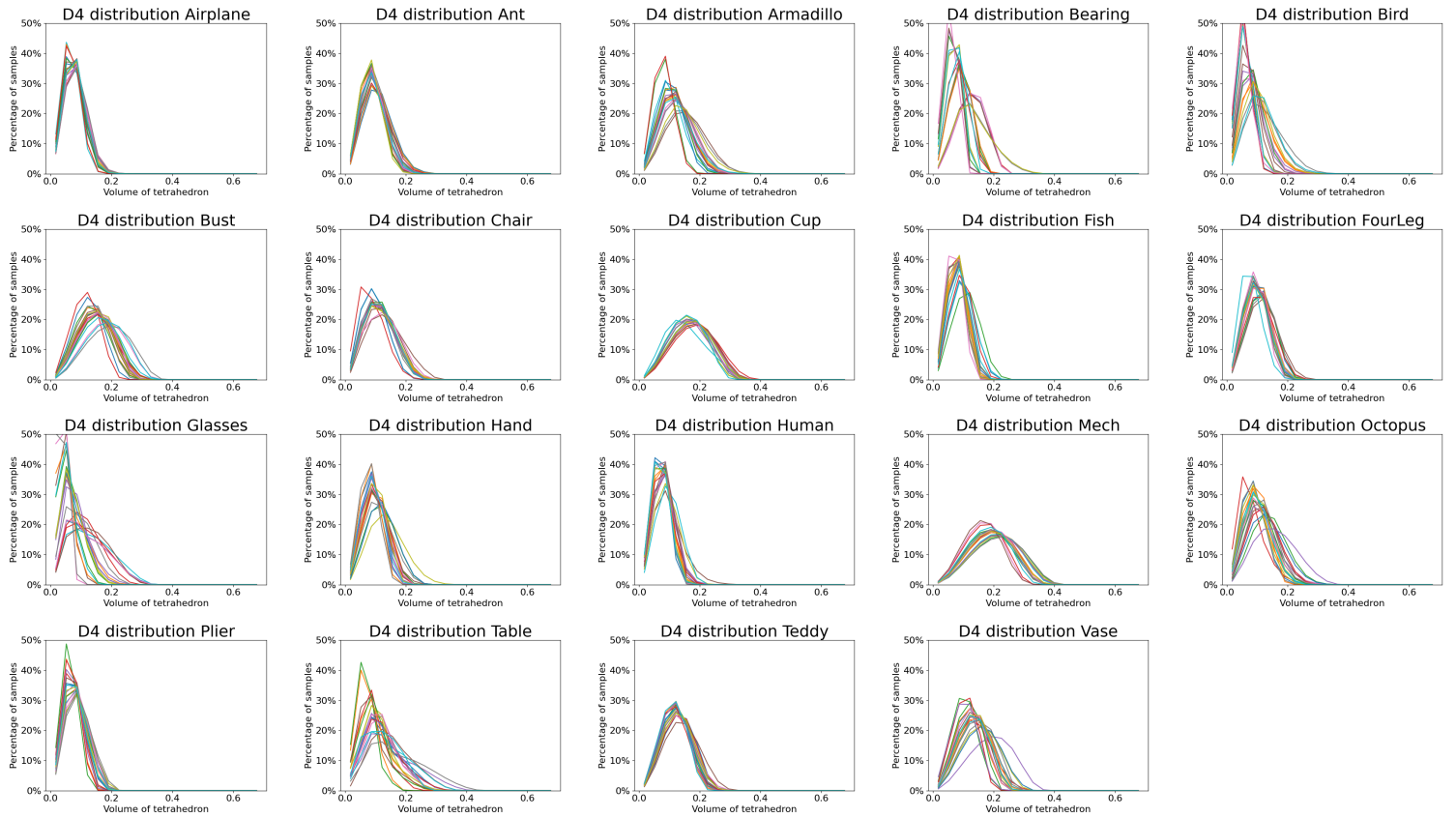


Figure A.65: D4 Distributions for each class in the database

Appendix B. Developer Notes

Appendix B.1. Report 1

As you can see from figure 1 it is already possible to select different render modes. And thus besides the flat-shading mode and wireframe mode we are also already able to visualize point cloud and convex hull. Much of the time in the first week has been spent on creating a good architecture (separated front-end and back-end) and finding out Open3D was suboptimal for GUI design (so had to create a GUI from scratch twice). Thus, the report has not been able to make it on Tuesday (I know it is mostly for ourselves).

Database reading writing with features has also been implemented, however the widget for visualizing the features not yet.

Appendix B.2. Report 2

The GUI has been greatly extended and improved as seen in the changed 1. It will not need major changes to architecture for the rest of the assignment, and new tabs can be easily created using re-usable widgets. We have tried 2 methods of resampling using Open3D, if it is not good enough. So we will explore using another library for that like PyMeshLab.

Translate + Alignment works, scaling is easy to finish and flip test is being worked on. The checks for translate, alignment, and scaling already exist. We will generate the pre- and post distribution plots after normalization pipeline is done.

Appendix B.3. Report 3

Normalization of vertices count is explored however there is not yet a histogram showing that the uniform distribution is uniform enough. Also it needs to get integrated into the pipeline after it is finished. 2D feature extractions are very fast. Histograms showing the correctness can only be generated after normalization is finished.

Appendix B.4. Report 4

Extra care has been put in submitting a decent quality report. ANN has been implemented but the final bit of 4 and 5 requires the histograms to be finished and EMD to be implemented. Silhouette hasnt been actively worked on, but should be doable in future to add to the application